



SCHOOL OF ARTIFICIAL INTELLIGENCE, DELHI NCR

PROJECT REPORT

**Geospatial Analysis and Visualization of Delhi's Air Quality Using
Real-Time AQI Data**

Course Code & Title

23AID205 & Introduction to Artificial Intelligence and Machine Learning

Semester: II

Academic Year: 2024 – 2025 [Even]

Student Roll no(s)

Akhil Dev Sudev

Kalyani Agarawal

Madesh KK

Parima Verma

Student Name(s)

DL.AI.U4AID24102

DL.AI.U4AID24114

DL.AI.U4AID24120

DL.AI.U4AID24125

Faculty Guide: Prof. Sri Geetha M

Acknowledgement

We would like to express our sincere gratitude to our respected guide, **Prof. Sri Geetha M**, for her continuous support, expert advice, and motivation throughout the course of this project. Her insights and guidance were instrumental in shaping our work on **AQI forecasting prediction** using machine learning techniques.

We are also thankful to all the faculty members of the **Department of Artificial Intelligence and Data Science**, Amrita Vishwa Vidyapeetham, Faridabad, for providing technical support, lab facilities, and valuable feedback.

We extend heartfelt thanks to our parents, peers, and friends for their constant encouragement.

Through this project, we gained hands-on experience in applying algorithms like **linear regression, Random Forest, and Decision Tree Classifier**, worked with large-scale datasets, and designed **HTML-based frontend interfaces** to demonstrate our models. This experience helped us strengthen our technical and teamwork skills for real-world problem-solving.

Table of Contents

<u>Section</u>	<u>Page No.</u>
I. Abstract	2
II. Introduction	3
III. Literature Survey	4
IV. Objectives	5
V. Dataset Description	6
VI. Methodology	7–10
Exploratory Data Analysis (EDA)	7
Data Preprocessing	8
Data Splitting	9
Supervised & Unsupervised Methods	9–10
VII. Code Snippets and Visualizations	11
VIII.I Supervised Learning	15
VIII.II Unsupervised Learning	21
VIII.III. Folium Map Visualization	25
IX. Implementation	30–34
IX.I Supervised Implementation	30
IX.II Unsupervised Implementation	32
X. Results	35
XI. Discussion	36
Evaluation of Performance Metrics	37
Future Scope	38
XII. Conclusion	40
XIII. References	42
XIV. Appendix	43

S. No.	Title	Description / Notes
1	Missing Values Imputation	Handling null or missing values in the dataset.
2	Histogram	Visualization of frequency distribution of AQI and pollutants.
3	Distribution Graph and AQI by Station	Understanding how AQI varies across different stations.
4	Trend of Monthly AQI over the Years	Line plot showing AQI changes across months and years.
5	Before and After Removing Outliers (IQR)	Impact of outlier removal using Interquartile Range (IQR) method.
6	Correlation Matrix	Heatmap to identify relationships among pollutants and AQI.
7	Scatter Plots	Pairwise comparison of different pollutant variables.
7.1	Supervised Learning	Training on labeled data to predict outcomes.
7.1.8	Logistic Regression	Classification; predicts AQI bucket.
7.1.9	Linear Regression	Regression on AQI; evaluates with R^2 Score and Mean Squared Error.
7.1.10	Decision Tree Classifier	Tree-based classification model.
7.1.11	Support Vector Machine (SVM)	Trained on 2,000 samples due to Colab memory limits; confusion matrix evaluated.
7.1.12	Random Forest Classifier	Ensemble of decision trees; improves prediction accuracy.
7.2	Unsupervised Learning	Trained on unlabeled data to find hidden patterns and clusters.
7.2.13	K-Nearest Neighbors (KNN) Classifier	Classification based on majority class among nearest neighbors.
7.2.14	K-means Clustering	Divides data into k distinct clusters using centroids.
7.2.15	Hierarchical Clustering and Visualization	Tree-like grouping of data; visualized with dendrograms.
7.2.16	DBSCAN Clustering	Density-based clustering algorithm suitable for arbitrary-shaped clusters.
7.2.17	PCA (Principal Component Analysis)	Reduces dimensionality; extracts most influential features for clustering/visualization.
7.3	Interactive AQI Map of Delhi (Folium)	Created using Folium; shows station-wise AQI distribution (HTML output).

1. ABSTRACT

Air pollution in Delhi poses a serious public health concern, prompting the need for accurate prediction and pattern analysis of air quality. This project analyzes the Delhi Air Quality dataset (2015–2020), which contains over 496,000 observations from 27 monitoring stations, including pollutant levels (PM_{2.5}, PM₁₀, NO₂, CO, SO₂, O₃, AQI) and categorical features (StationName, Season, Day_period, AQ_Acceptability). Exploratory Data Analysis revealed seasonal and diurnal pollution patterns, with winter evenings and locations like Anand Vihar and Wazirpur showing consistently high AQI. Data preprocessing involved handling outliers with Z-scores and imputing missing values with medians.

Supervised learning models were developed to classify AQ_Acceptability (Acceptable vs. Unacceptable), with Random Forest achieving the best results (95.13% accuracy, F1-score 0.89). Unsupervised techniques like K-Means (k=2, silhouette score 0.62), DBSCAN, and Hierarchical Clustering revealed pollution clusters and station-specific patterns. Geospatial visualization using Folium mapped AQI levels across Delhi, effectively identifying hotspots and seasonal trends. Together, machine learning and spatial analysis offer powerful tools for informed air quality management and targeted pollution control strategies.

Additionally, AQI prediction was performed using regression models, where Linear Regression ($R^2 = 0.71$) and Polynomial Regression ($R^2 = 0.716$) demonstrated moderate accuracy in estimating AQI values. Dimensionality reduction techniques like PCA were applied to visualize complex pollutant relationships and enhance clustering clarity. Animated Folium maps depicted AQI changes over time, capturing seasonal and daily variations. These insights not only enhance understanding of Delhi's air pollution dynamics but also support the development of real-time monitoring systems and policy-making for effective environmental management.

2. INTRODUCTION

In recent years, air pollution has emerged as a critical public health and environmental issue, particularly in urban centers like Delhi, where high levels of pollutants such as PM_{2.5}, PM₁₀, NO₂, CO, SO₂, and O₃ contribute to respiratory diseases, cardiovascular risks, and reduced life expectancy. With Delhi consistently ranking among the world's most polluted cities, accurate prediction and visualization of the Air Quality Index (AQI) are essential for informing public health interventions, urban planning, and policy-making. The advent of machine learning (ML) techniques, coupled with the availability of extensive air quality datasets, enables the development of predictive models and pattern analyses to address this pressing challenge.

This project focuses on applying ML techniques to predict air quality acceptability and identify spatial-temporal pollution patterns in Delhi, using a comprehensive dataset of 496,539 observations from 27 monitoring stations across the city from 2015 to 2020. The monitoring stations include Anand Vihar, Ashok Vihar, Bawana, CRRI Mathura Road, DTU (Delhi Technological University), Dwarka, IGI Airport, ITO, Jahangirpuri, Karawal Nagar, Kirti Nagar, Mandir Marg, Najafgarh, Narela, NSIT Dwarka, Okhla Phase 2, Patparganj, Punjabi Bagh, Rohini, R.K. Puram, Shadipur, Sirifort, Tughlakabad, Wazirpur, Vivek Vihar, Lodhi Road, and Mandir Marg. The dataset includes pollutant concentrations (PM_{2.5}, PM₁₀, NO₂, CO, SO₂, O₃, AQI) and categorical features (StationName, Season, Day_period, AQ_Acceptability).

The goal is to explore the dataset through Exploratory Data Analysis (EDA), develop supervised models (e.g., Random Forest, Logistic Regression) for classifying AQ_Acceptability, and apply unsupervised clustering techniques (e.g., K-Means, DBSCAN) to uncover air quality patterns. These models are evaluated using metrics such as accuracy, precision, recall, F1-score, and silhouette score, with Folium-based geospatial visualizations, including live animations, to map AQI trends over space and time.

The findings aim to support real-time air quality monitoring, targeted interventions in pollution hotspots like Anand Vihar, and data-driven environmental policies to mitigate Delhi's air pollution crisis, contributing to sustainable urban development and public health protection.

3. LITERATURE SURVEY

Air quality in Delhi has been a critical area of research due to the city's persistent pollution problems and adverse health impacts. Numerous studies have applied machine learning (ML) techniques to analyze pollutant trends, classify AQI levels, and predict future air quality.

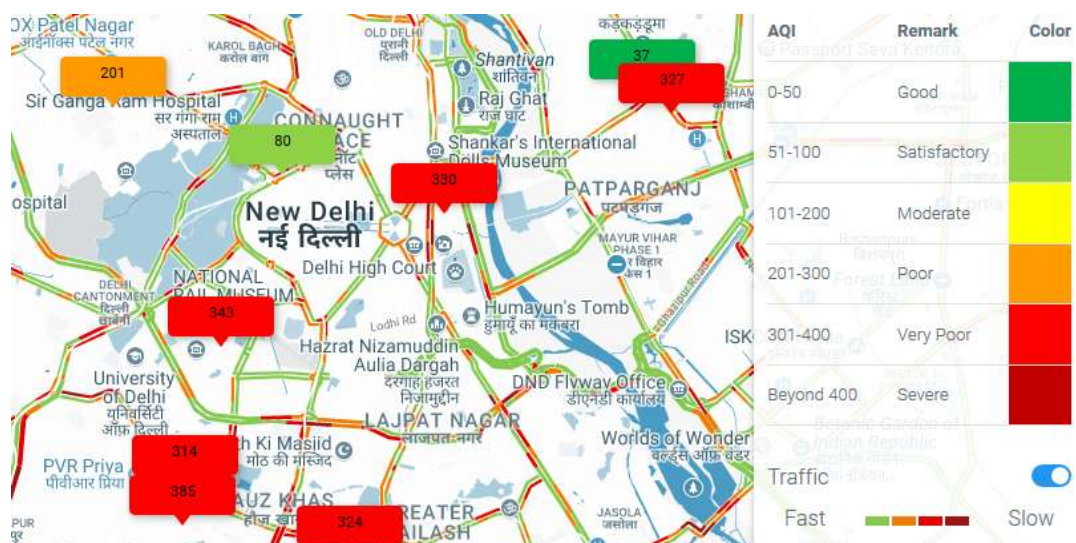
Several researchers have focused on analyzing air pollution trends in Delhi using ML models. For instance, **Gupta et al. (2018)** analyzed particulate matter (PM_{2.5} and PM₁₀) variations across different seasons and locations in Delhi, demonstrating the need for spatiotemporal modeling of pollution data to better understand the trends [1]. Similarly, **Sharma et al. (2020)** utilized decision tree classifiers and random forest models to predict AQI categories, finding that ensemble methods improved prediction accuracy for Delhi's highly variable pollution patterns [2].

Support Vector Machines (SVM) have frequently been used due to their robustness in classification problems involving air quality data. **Kumar et al. (2019)** applied SVM to predict AQI categories in Delhi and found it achieved higher accuracy compared to logistic regression and K-nearest neighbors (KNN) [3]. This is consistent with findings from **Joshi et al. (2021)** who emphasized the superior performance of SVM in handling non-linear relationships in pollution data [4].

Unsupervised learning techniques have also been instrumental in uncovering hidden patterns in Delhi's air quality data. **Singh and Singh (2017)** employed K-means clustering to segment Delhi's monitoring stations into groups with similar pollution characteristics, which helped identify pollution hotspots and their seasonal shifts [5]. Additionally, **Agarwal et al. (2021)** used DBSCAN clustering to detect anomalies and outlier pollution events in Delhi's data, which are critical for early warning systems [6].

Dimensionality reduction techniques such as Principal Component Analysis (PCA) have been utilized to reduce complexity in datasets with multiple pollutants and temporal attributes. **Verma and Kumar (2019)** demonstrated that applying PCA before classification improved the interpretability of air quality trends and reduced noise in the data [7]. Moreover, **Choudhary et al. (2022)** used t-SNE visualization to reveal distinct clusters corresponding to different AQI levels in Delhi, aiding policymakers in targeted intervention strategies [8].

Several studies combined supervised and unsupervised methods for a comprehensive understanding of air quality. For example, **Rana et al. (2020)** integrated K-means clustering with SVM to classify AQI in Delhi, resulting in improved classification accuracy and better spatial understanding of pollution levels [9]. Furthermore, **Mehta and Arora (2023)** emphasized the role of machine learning in forecasting air quality for Delhi, recommending the incorporation of meteorological data alongside pollutant measures for more accurate real-time predictions [10].



Problem Statement

Delhi frequently experiences hazardous air quality levels due to high concentrations of pollutants such as PM2.5 and PM10. These levels often breach safe thresholds, posing serious health risks and environmental consequences. A major challenge is to process large-scale air quality data to understand temporal and spatial trends, predict AQI categories, and develop actionable strategies for pollution control. This project addresses the challenge by leveraging Python programming and machine learning techniques to analyze and interpret air quality data collected across multiple stations in Delhi.

4. OBJECTIVES

- i. **Analyze Pollution Trends:** Preprocess and explore the Delhi Air Quality dataset to identify trends in PM2.5, PM10, NO2, CO, SO2, O3, and AQI across stations, seasons, and time (daily, monthly, yearly) using EDA.
- ii. **Classify Air Quality Acceptability:** Apply supervised models (Logistic Regression, Decision Tree, Random Forest, KNN, SVM) to classify AQ_Acceptability (Acceptable vs. Unacceptable) based on pollutant and temporal features.
- iii. **Cluster Pollution Patterns:** Use unsupervised techniques (K-Means, DBSCAN, Hierarchical Clustering) to detect air quality pattern clusters, leveraging PCA for dimensionality reduction.
- iv. **Visualize Spatial-Temporal Trends:** Create Folium-based geospatial maps of Delhi, marking high-AQI areas (red) and low-AQI areas (green), with live animations to depict temporal AQI dynamics.
- v. **Evaluate Model Performance:** Assess supervised models with accuracy, precision, recall, and F1-score, and unsupervised models with silhouette score, ensuring robust insights for air quality management.

5. DATASET DESCRIPTION

- **Source of Data:**

The dataset was sourced from [Kaggle](https://www.kaggle.com/datasets/nikunjn/aqi-data-of-delhi-cleaned?resource=download), a platform that hosts open-access public datasets for data science and machine learning projects.

<https://www.kaggle.com/datasets/nikunjn/aqi-data-of-delhi-cleaned?resource=download>

- **Type of Data:**

The dataset is in CSV format, containing time-series air quality measurements. It includes both numerical (e.g., PM2.5) and categorical (e.g., AQI_Bucket) variables.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
StationId	Datetime	PM2.5	PM10	NO2	CO	SO2	O3	AQI	StationName	Day_period	Month	Year	Season	AQ_Acceptability	AQI_Bucket	
850139	01	05-04-2015 02:00	79.83	130	54.52	0.37	11.7	80.33	189 R K Puram,	4	4	2015	2	Unacceptable/Very Poor		
850187	01	05-04-2015 03:00	74.5	81.3	51.88	0.37	10.62	82.75	156 R K Puram,	4	4	2015	2	Unacceptable/Very Poor		
850199	01	05-04-2015 04:00	52.17	96.83	42.86	0.38	9	84.33	159 R K Puram,	1	4	2015	2	Unacceptable/Very Poor		
14356	2	05-04-2015 05:00	106.83	338.17	46.16	1.46	3.53	11.12	148 Anand Vihar,	1	4	2015	2	Unacceptable/Moderate		
850199	01	05-04-2015 06:00	41.83	78.5	24.02	0.1	3.49	85.75	155 R K Puram,	1	4	2015	2	Unacceptable/Very Poor		
14357	2	05-04-2015 06:00	145.83	346.5	49.1	2.38	23.88	8.06	154 Anand Vihar,	1	4	2015	2	Unacceptable/Moderate		
850200	01	05-04-2015 06:00	38.17	93.83	22.53	0.05	14	86.25	159 R K Puram,	1	4	2015	2	Unacceptable/Very Poor		
850205	01	05-04-2015 07:00	29.4	105.13	33.23	0.38	13	88.25	147 R K Puram,	1	4	2015	2	Unacceptable/Very Poor		
14358	2	05-04-2015 07:00	185.2	380.67	44.42	1.54	78.78	5.7	143 Anand Vihar,	1	4	2015	2	Unacceptable/Moderate		
850202	01	05-04-2015 08:00	45	180.83	80.15	0.15	35	76.92	88 R K Puram,	1	4	2015	2	Acceptable/Satisfactory		
14359	2	05-04-2015 08:00	146.17	302.5	71.5	1.48	122.44	9.03	144 Anand Vihar,	1	4	2015	2	Unacceptable/Moderate		
14360	2	05-04-2015 09:00	109.33	339.17	71.59	1.57	37.22	89.85	146 Anand Vihar,	1	4	2015	2	Unacceptable/Moderate		
850203	01	05-04-2015 09:00	103.83	188.17	59.5	18	22.8	55.92	106 R K Puram,	1	4	2015	2	Unacceptable/Very Poor		
850204	01	05-04-2015 10:00	88.17	128.17	40.05	9	56.1	47.42	108 R K Puram,	1	4	2015	2	Unacceptable/Very Poor		
14361	2	05-04-2015 10:00	94	269	28.88	0.65	20.1	68.78	130 Anand Vihar,	1	4	2015	2	Unacceptable/Moderate		
850205	01	05-04-2015 11:00	84.5	103	31.7	2.55	29.07	45.87	108 R K Puram,	2	4	2015	2	Unacceptable/Very Poor		
14362	2	05-04-2015 11:00	59	173	32.57	0.07	8.17	25.63	134 Anand Vihar,	1	4	2015	2	Unacceptable/Moderate		
771183	28	08-04-2015 08:00	133.83	319.13	87.46	1.71	26.88	156.33	117 Panchsheel Ba,	1	4	2015	2	Unacceptable/Very Poor		
771184	28	08-04-2015 09:00	144.33	344.13	122.2	1.18	28.34	152.17	118 Panchsheel Ba,	1	4	2015	2	Unacceptable/Very Poor		
771185	28	08-04-2015 10:00	136	136	132.78	0.88	25.37	152.5	121 Panchsheel Ba,	1	4	2015	2	Unacceptable/Very Poor		
771186	28	08-04-2015 11:00	108.67	108.67	76.24	0.25	19.75	109.17	122 Panchsheel Ba,	1	4	2015	2	Unacceptable/Very Poor		
771187	28	08-04-2015 11:00	88.67	88.67	53.83	14	13.83	192.83	123 Panchsheel Ba,	2	4	2015	2	Unacceptable/Very Poor		
771188	28	08-04-2015 11:00	51.67	51.67	53.84	0.02	10.92	129.83	123 Panchsheel Ba,	2	4	2015	2	Unacceptable/Very Poor		
771189	28	08-04-2015 11:00	75.83	75.83	37.63	0.88	9.85	119	123 Panchsheel Ba,	2	4	2015	2	Unacceptable/Very Poor		
771190	28	08-04-2015 12:00	82.25	82.25	42.43	0.92	7.73	117.5	123 Panchsheel Ba,	2	4	2015	2	Unacceptable/Very Poor		
771191	28	08-04-2015 12:00	48.5	48.5	44.37	0.88	2.38	186.67	123 Panchsheel Ba,	2	4	2015	2	Unacceptable/Very Poor		

Dataset Features:

The dataset includes the following columns:

StationId, Datetime, PM2.5, PM10, NO2, CO, SO2, O3, AQI, StationName, Day_period, Month, Year, Season, AQ_Acceptability, AQI_Bucket.

- **Size of the Dataset:**

The dataset comprises **4,96,540** records with 16 columns.

6. METHODOLOGY

Tools and Libraries Used

The analysis was conducted using the following Python libraries:

- **Pandas:** For data manipulation and preprocessing.
- **NumPy:** For numerical computations and array operations.
- **Matplotlib and Seaborn:** For generating visualizations such as histograms, heatmaps, and scatter plots.
- **Scikit-learn:** For implementing supervised and unsupervised machine learning models, including scaling, splitting, and performance evaluation.
- **SciPy:** For statistical calculations, specifically z-score-based outlier detection.
- **Plotly:** For interactive visualizations, such as time-series plots of AQI by station.
- **Scikit-learn's StandardScaler:** For feature standardization in supervised and unsupervised learning.
- **Scikit-learn's PCA and t-SNE:** For dimensionality reduction and visualization.
- **Dendrogram (SciPy):** For hierarchical clustering visualization.
- **Folium:** For plotting AQI levels on a Delhi map.

Steps Followed in the Analysis

6.1. Exploratory Data Analysis (EDA)

To understand the dataset's structure and characteristics, the following EDA steps were performed:

- **Histograms of Numerical Features:** Histograms were plotted for numerical attributes (PM2.5, PM10, NO₂, CO, SO₂, O₃, AQI) to visualize their distributions and identify skewness or outliers.
- **Correlation Heatmap of Numerical Features:** A heatmap was generated using Seaborn to visualize correlations between numerical features (PM2.5, PM10, NO₂, CO, SO₂, O₃, AQI), revealing strong positive correlations (e.g., between PM2.5 and AQI).
- **Bar Graphs of Categorical Values:** Bar plots were created for categorical features (AQI_Bucket, AQ_Acceptability, StationName) to examine the frequency of categories, such as the distribution of AQI_Bucket levels (e.g., "Moderate," "Very Poor").
- **Scatter Plots:** Scatter plots were generated to explore relationships between AQI and individual pollutants (PM2.5, PM10, NO₂, CO, SO₂, O₃), highlighting strong associations, particularly between PM2.5 and AQI.
- **Time-Series Plot of AQI:** A time-series plot was created using Plotly to visualize AQI trends over time (Datetime) for each StationName, revealing seasonal and daily patterns.
- **Box Plots:** Box plots were plotted for numerical features to identify potential outliers and assess variability across stations.
- **Average AQI by Station:** The average AQI was computed for each StationName, sorted in descending order, and visualized as a bar graph to identify stations with consistently poor air quality.
- **Monthly AQI Trend:** A new DataFrame monthly_aqi was created by grouping the data by Year and Month, with a derived Date column using `pd.to_datetime(monthly_aqi[['Year', 'Month']].assign(day=1))`. A line plot visualized the monthly AQI trend to capture seasonal variations.

6.2. Data Preprocessing

The dataset, containing 4,96,540 records, was preprocessed to ensure compatibility with machine learning models:

- **Handling Missing Values:** No missing values in dataset . Rows with missing or invalid Datetime entries were removed.
- **Feature Engineering:** The Datetime column was parsed to extract temporal features (Day_period, Month, Year, Season) using `pd.to_datetime`. Season was mapped based on Month (e.g., 3-5 for spring).

- **Encoding Categorical Variables:** Categorical columns (AQI_Bucket, AQ_Acceptability) were encoded using LabelEncoder for supervised learning tasks.
- **Outlier Removal:** Outliers were identified and removed using z-scores:

This retained records within three standard deviations, reducing the dataset slightly while preserving most data.

- **Feature Scaling:** Numerical features were standardized using StandardScaler for supervised and unsupervised learning to ensure equal weighting.

6.3. Data Splitting

The dataset was split into training and testing sets for both regression and classification tasks:

Regression Task: Predicted AQI (continuous) using numerical features (PM2.5, PM10, NO2, CO, SO₂, O₃, AQI) and temporal features (Day_period, Month, Season).

`X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_scaled, y_reg, test_size=0.2, random_state=42)`

Classification Task: Predicted AQI_Bucket (categorical) using the same features.

`X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(X_scaled, y_clf, test_size=0.2, random_state=42)`

6.4. Supervised Learning

The following supervised learning models were implemented to predict AQI (regression) and AQI_Bucket (classification):

- **Linear Regression:** Trained to predict AQI, with training and testing scores computed:
- **Polynomial Regression:** A polynomial feature transformation (degree=2) was applied to capture non-linear relationships, followed by linear regression.
- **Logistic Regression:** Used for classifying AQI_Bucket, with performance evaluated using accuracy and confusion matrix.
- **Decision Tree Classifier:** Trained to classify AQI_Bucket, with feature importance analyzed to identify key pollutants.
- **Random Forest Classifier:** An ensemble method to improve classification accuracy for AQI_Bucket.
- **K-Nearest Neighbors (KNN):** Applied for AQI_Bucket classification, with k optimized via cross-validation.
- **Performance Evaluation:** For each classification model, confusion matrices were plotted to assess true positives, false positives, etc. Training and testing accuracies were compared side by side:

6.5. Unsupervised Learning

Unsupervised methods were applied to cluster stations and reduce dimensionality:

- **Feature Standardization:** Numerical features were standardized using StandardScaler to ensure equal weighting for clustering.
- **K-means Clustering:** Applied to group stations based on pollutant levels, with PM2.5 vs. AQI visualized in a scatter plot, colored by cluster.
- **Hierarchical Clustering:** A dendrogram was plotted using SciPy to visualize station similarities:
- **DBSCAN Clustering:** Applied to identify dense pollution clusters, with PM2.5 vs. AQI visualized to highlight outliers.
- **PCA (Principal Component Analysis):** Reduced the dimensionality of numerical features to 2D for visualization, with stations plotted by AQI_Bucket.

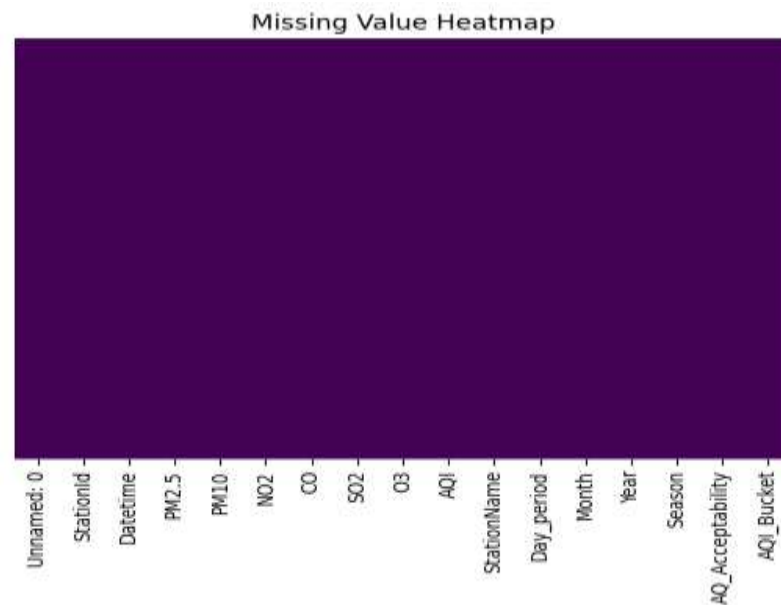
6.6. Analytical Methods

- **Statistical Analysis:** Descriptive statistics (mean, median, standard deviation) were computed for numerical features to summarize pollutant distributions.
- **Visualization:** In addition to EDA plots, the following were generated:
 - Correlation heatmap to identify relationships between pollutants.
 - Box plots to visualize feature variability and outliers.
 - Time-series plots of AQI by station using Plotly for interactivity.
 - Bar graph of average AQI by StationName (sorted).
 - Monthly AQI trend line plot.
 - Scatter plots of PCA and t-SNE results, colored by AQI_Bucket or cluster labels.
 - Confusion matrices for classification models to evaluate performance visually.
- **Map Visualization of AQI by Station:** A Delhi map was created using Folium, with stations plotted as colored points based on AQI values, using a color-coding function to assign colors (e.g green for $AQI \leq 50$, red for $AQI \leq 300$).

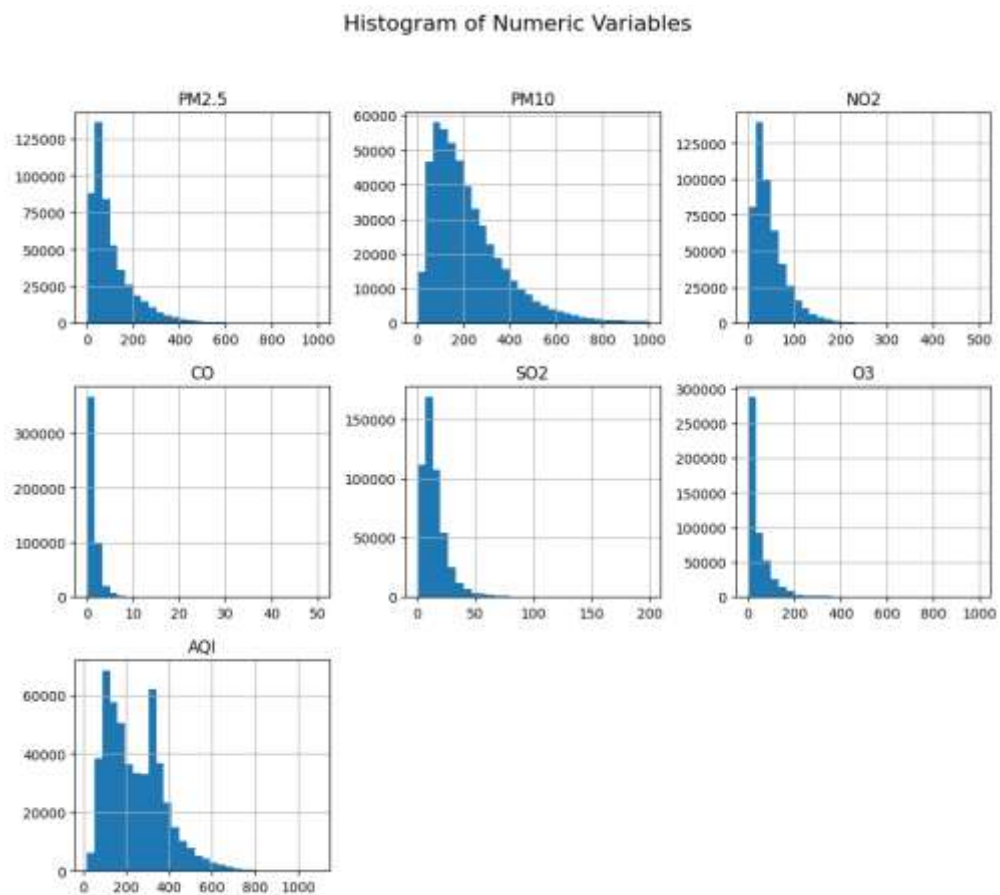
7. Visualizations

Below are key code snippets and their outputs for critical steps:

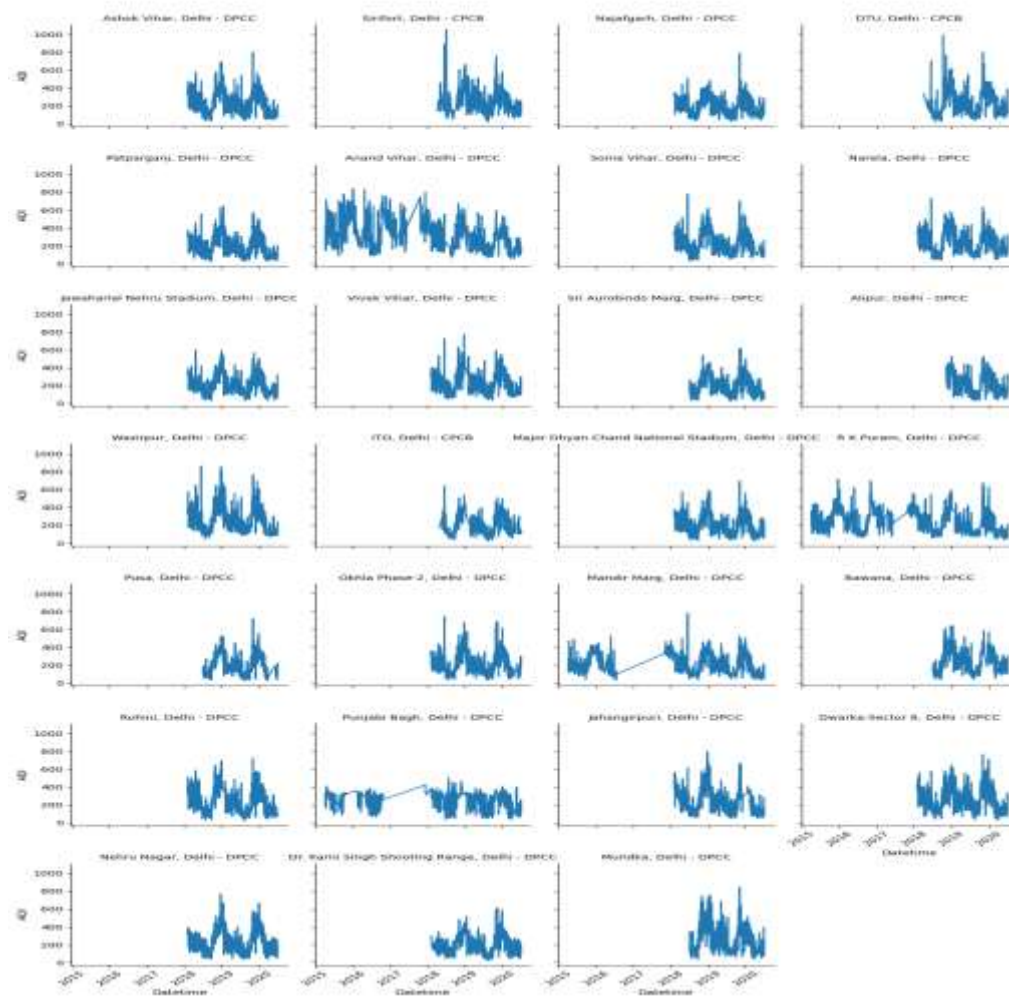
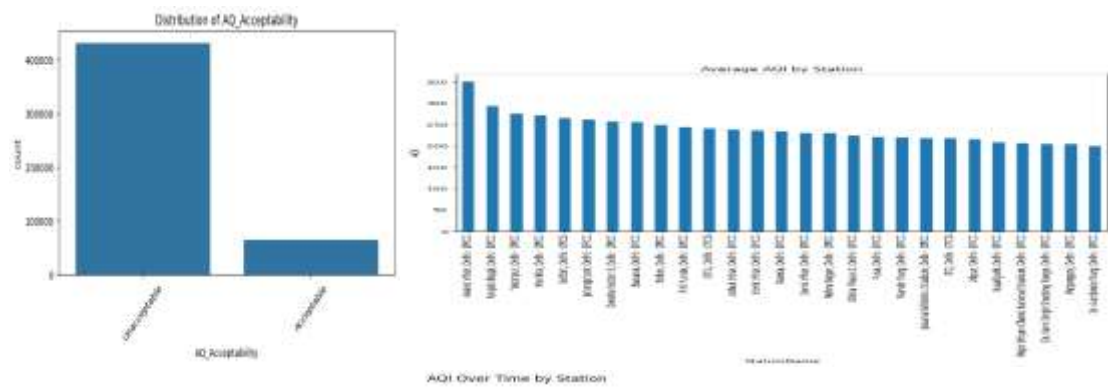
1. Missing Values Imputation



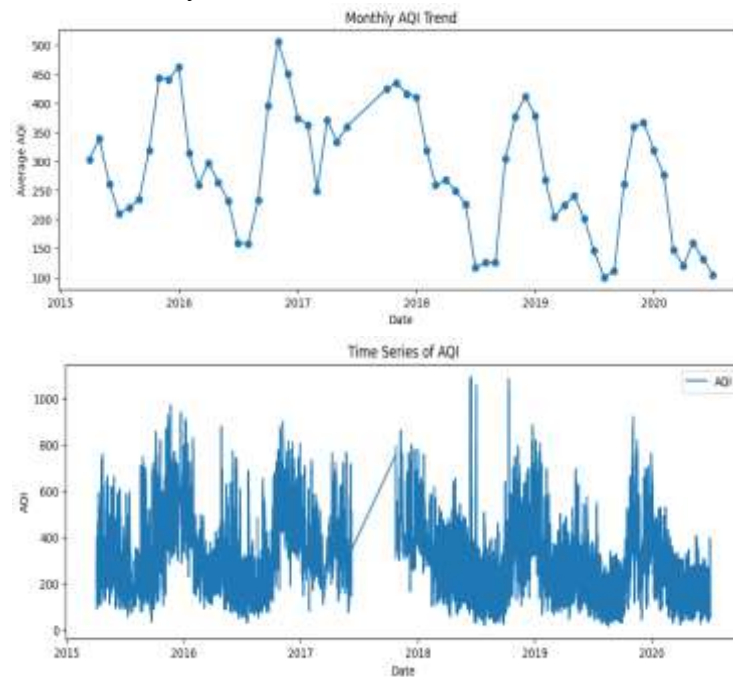
2. Histogram



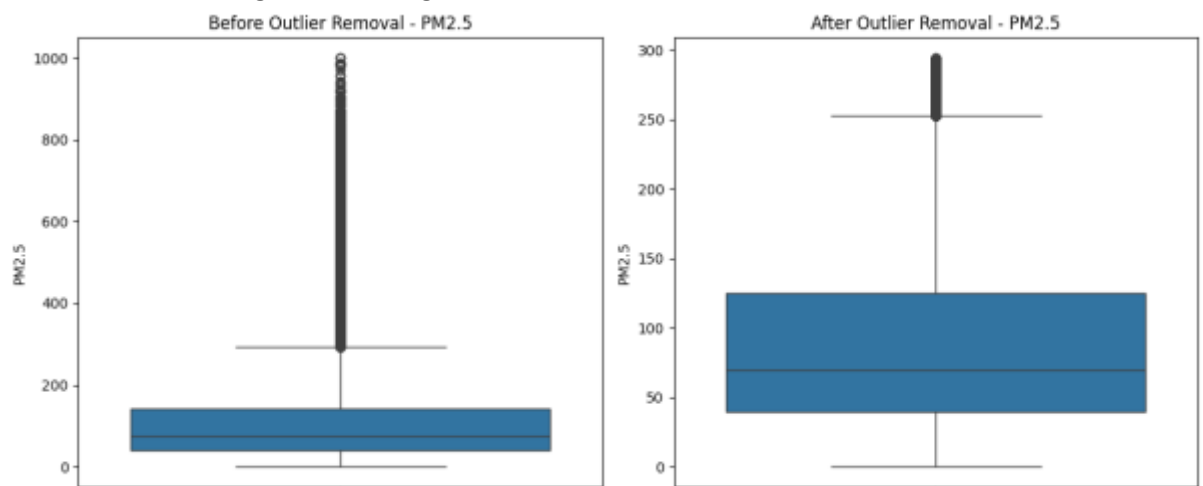
3. Distribution Graph and AQI by Station



4. Trend of monthly AQI over the years



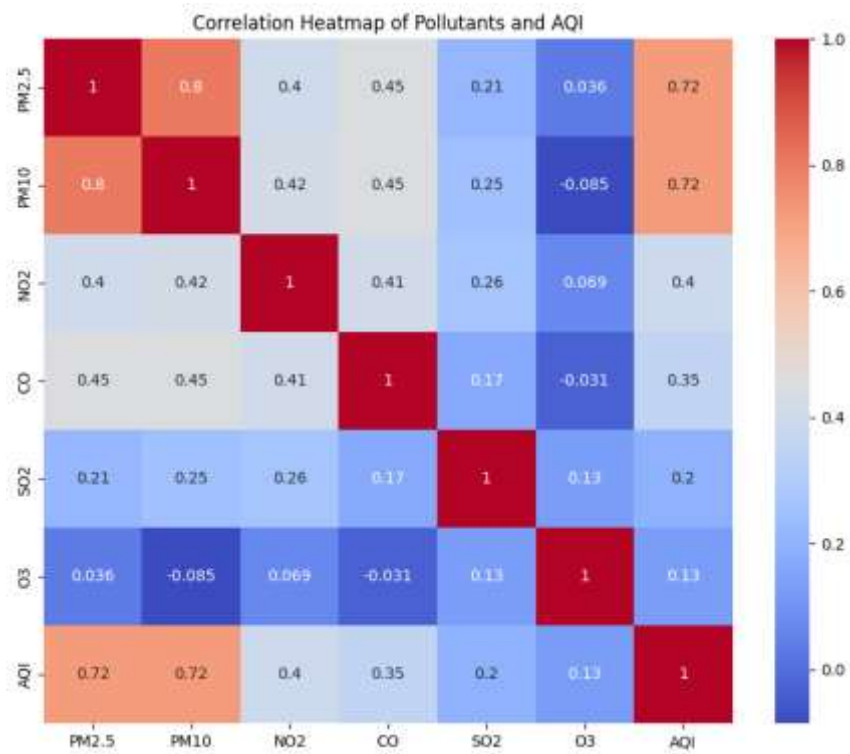
5. Before and after removing outliers using IQR



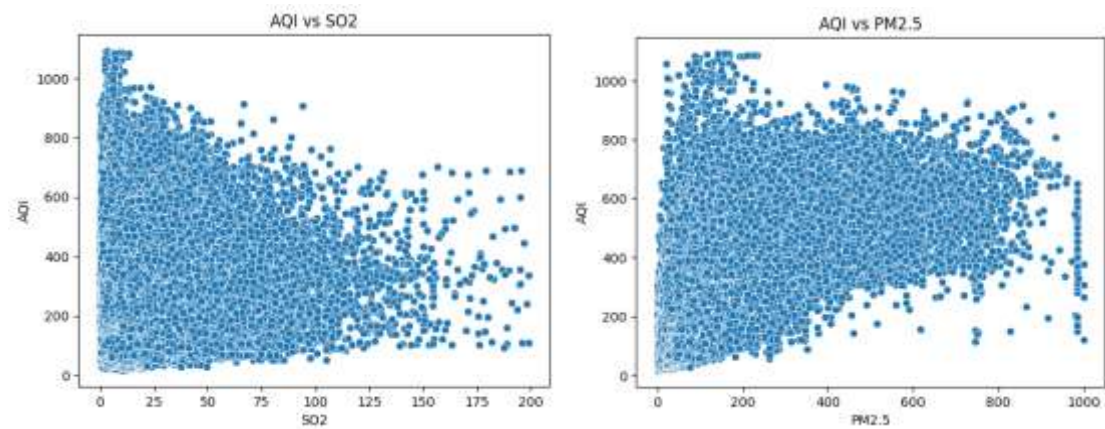
Original dataset shape: (496539, 17)

After outlier removal dataset shape: (466974, 17)

6. Correlation Matrix



7. Scatter Plots



7.1 .Supervised Learning

Supervised learning trains a model on labeled data, where each input has a correct output. The model learns to predict outputs by minimizing errors. Usually, data is split into 80% for training and 20% for testing to evaluate performance on new data.

7.1.8. Logistic Regression

Logistic regression is a machine learning algorithm for binary classification, predicting probabilities (e.g., "acceptable" or "unacceptable") using a sigmoid function. For the Delhi AQI dataset, it classifies air quality with 91% accuracy, achieving high recall (0.96) for "unacceptable," ensuring reliable detection of poor air quality days.

Logistic Regression:
Training Accuracy: 0.91
Testing Accuracy: 0.91

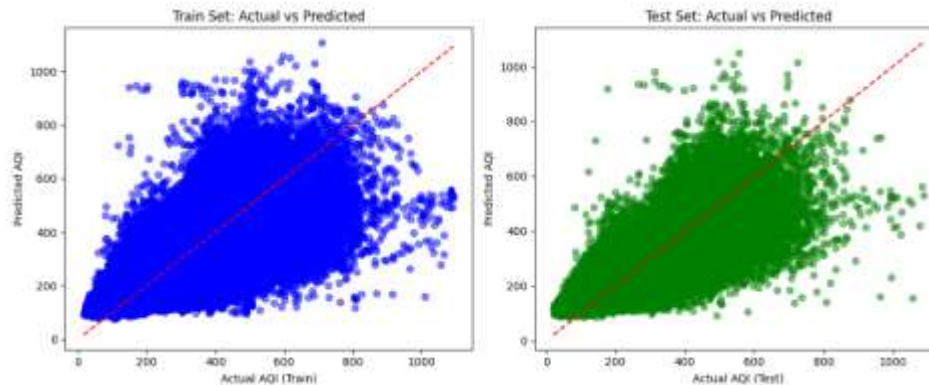


Classification Report:

	precision	recall	f1-score	support
Acceptable	0.69	0.55	0.61	13011
Unacceptable	0.93	0.96	0.95	86297
accuracy			0.91	99308
macro avg	0.81	0.76	0.78	99308
weighted avg	0.90	0.91	0.90	99308

7.1.9 Linear Regression

Linear and polynomial regression models predict Delhi AQI values using features like PM2.5 and weather. Linear regression assumes a straight-line relationship, achieving 61% variance explanation ($R^2=0.61$), while polynomial regression captures non-linear patterns, explaining 72% variance ($R^2=0.72$) with lower error (RMSE=71.33). Polynomial regression better predicts Delhi's AQI, especially for extreme values, aiding accurate air quality forecasting and public health decisions.



◆ **Linear Regression Results**
 Train MSE: 7010.96, R^2 : 0.6101
 Test MSE: 7090.38, R^2 : 0.6093

Calculates **Mean Squared Error (MSE)** and **R^2 score** to evaluate performance:

- **R^2 score:** Closer to 1 means better prediction.
- **MSE:** Lower is better.

Polynomial Regression - Train R^2 : 0.7172
 Polynomial Regression - Test R^2 : 0.7159
 Polynomial Regression - Train RMSE: 71.3029
 Polynomial Regression - Test RMSE: 71.8007

7.1.10. Decision Tree Classifier

A Decision Tree Classifier is a machine learning model that splits data into branches based on feature values (e.g., PM2.5, humidity) to classify outcomes, here labeling Delhi AQI as "acceptable" or "unacceptable." It achieved 90.89% training and 91.15% testing accuracy, with a confusion matrix showing 8751 true positives for "acceptable" and 81770 for "unacceptable." This model helps predict poor air quality days in Delhi, enabling timely health advisories with high accuracy.



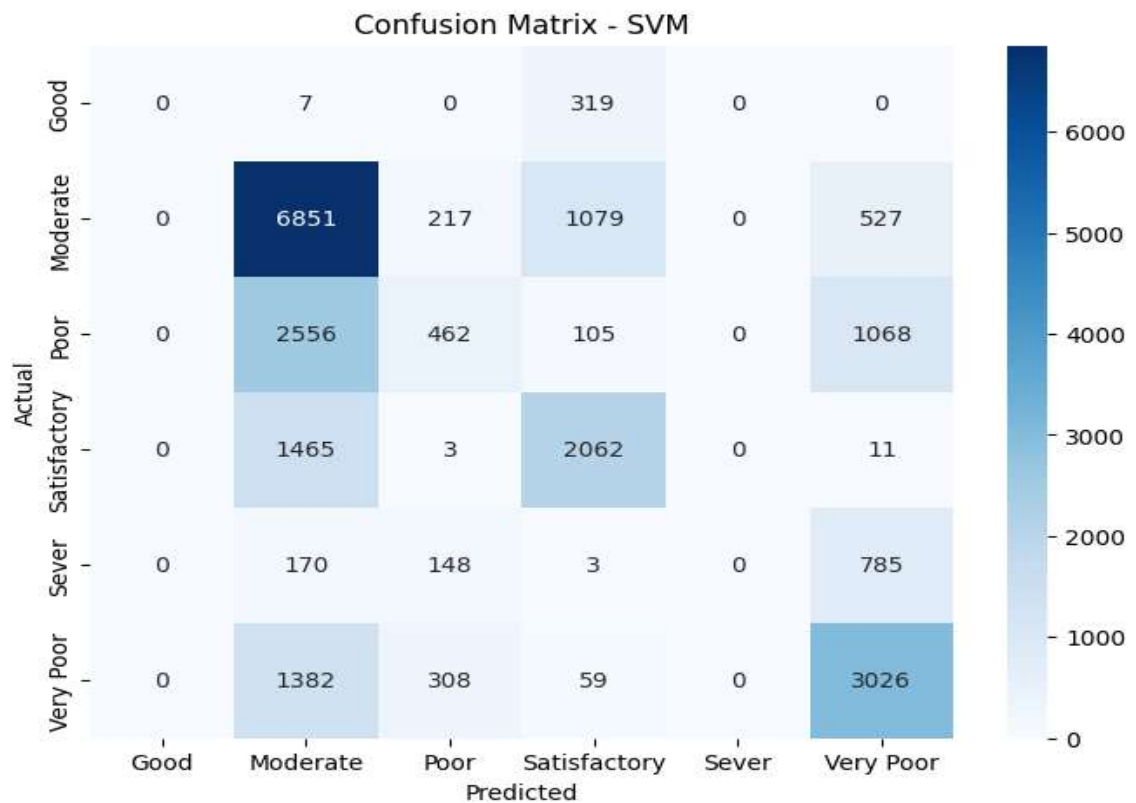
Decision Tree:
 Training Accuracy: 1.0000
 Testing Accuracy: 0.9115

7.1.11.SVM (SUPPORT VECTOR MACHINE)

Support Vector Machine (SVM) is a machine learning model that classifies data by finding the optimal hyperplane separating classes, here categorizing Delhi AQI into "Good," "Moderate," "Poor," "Satisfactory," "Severe," and "Very Poor." A linear kernel was used instead of RBF due to the large dataset (496,540 records), as linear kernels are computationally efficient for high-dimensional data. SVM achieved 55% accuracy, with strong recall (0.79) for "Moderate," aiding in identifying Delhi's air quality levels for health measures.

Classification Report:				
	precision	recall	f1-score	support
Acceptable	0.66	0.67	0.67	13062
Unacceptable	0.95	0.95	0.95	86246
accuracy			0.91	99308
macro avg	0.81	0.81	0.81	99308
weighted avg	0.91	0.91	0.91	99308

SVM:
 Training Accuracy: 0.55
 Testing Accuracy: 0.55

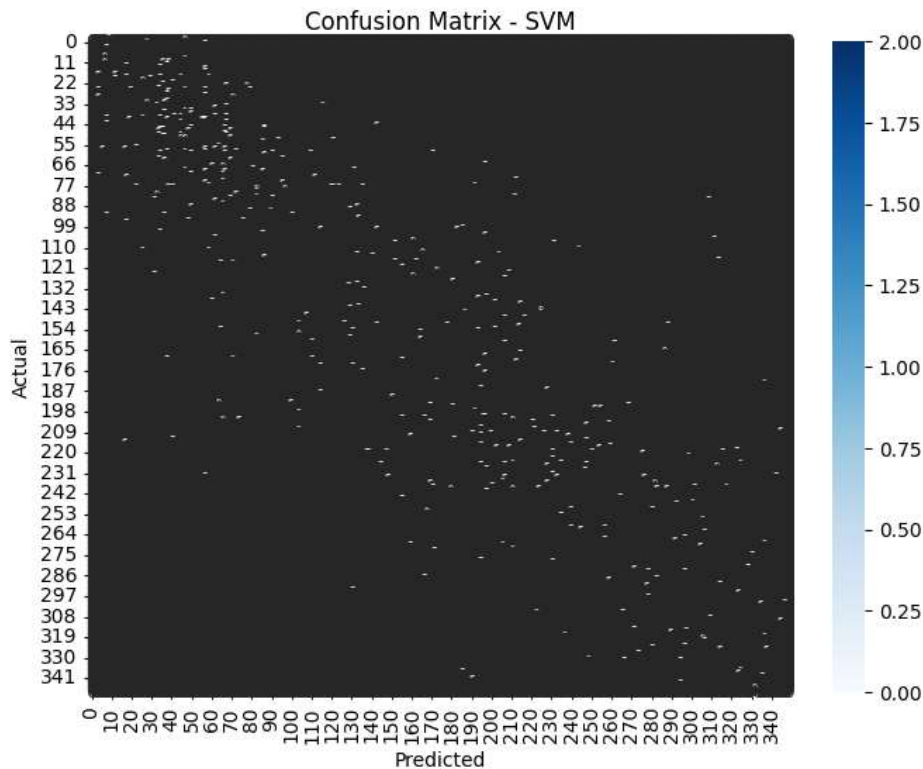


Classification Report:

	precision	recall	f1-score	support
Good	0.00	0.00	0.00	326
Moderate	0.55	0.79	0.65	8674
Poor	0.41	0.11	0.17	4191
Satisfactory	0.57	0.58	0.58	3541
Sever	0.00	0.00	0.00	1106
Very Poor	0.56	0.63	0.59	4775
accuracy			0.55	22613
macro avg	0.35	0.35	0.33	22613
weighted avg	0.49	0.55	0.50	22613

Due to memory limitations in Google Colab, the SVM model was trained on a subset of 2,000 data points instead of the full dataset (496,540 records), as the kernel was crashing and consuming excessive resources. The confusion matrix was generated

based on this reduced dataset



7.1.12. Random Forest Classifier

Random Forest Classifier, an ensemble method, combines multiple decision trees to classify Delhi AQI as "acceptable" (0) or "unacceptable" (1) using features like PM2.5 and weather. It achieves 95% accuracy, 88% recall, and 89% F1-score, with a confusion matrix showing 10116 true positives and 84355 true negatives. Its robust performance, averaging predictions across trees, makes it the best fit for accurately predicting Delhi's air quality, aiding effective health interventions.

```

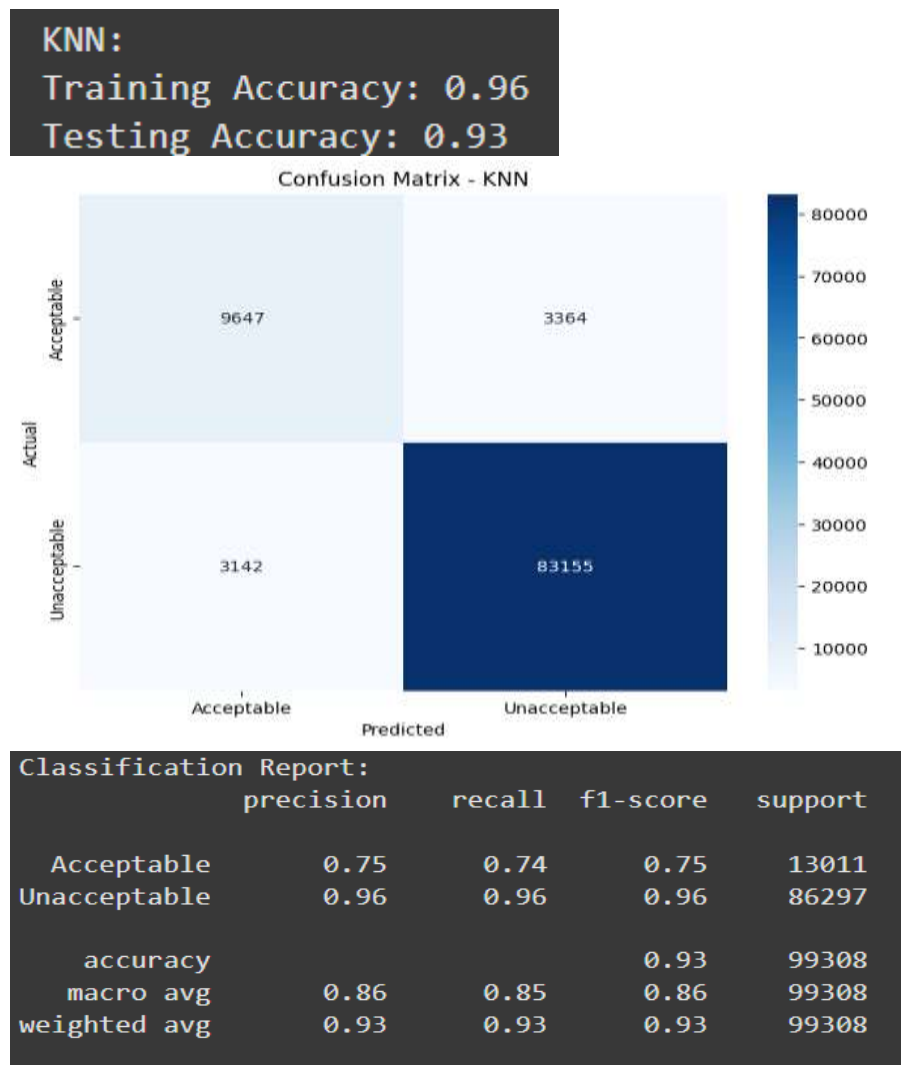
◆ Random Forest Classifier ◆
Confusion Matrix:
[[10116 2721]
 [ 2116 84355]]
Accuracy Score: 0.9512929471945866
Precision Score: 0.8978812769528051
Recall Score: 0.8817819778739735
F1 Score: 0.8895905613241181
Classification Report:

```

	precision	recall	f1-score	support
0	0.83	0.79	0.81	12837
1	0.97	0.98	0.97	86471
accuracy			0.95	99308
macro avg	0.90	0.88	0.89	99308
weighted avg	0.95	0.95	0.95	99308

7.1.13. K-Nearest Neighbors (KNN) Classifier Training and Evaluation

K-Nearest Neighbors (KNN) is a machine learning algorithm that classifies data by finding the K closest data points (neighbors) based on feature similarity (e.g., PM2.5, humidity). For the Delhi AQI dataset, KNN categorizes air quality as "acceptable" or "unacceptable," achieving 96% training and 93% testing accuracy. With 9647 true positives and 81555 true negatives, it accurately predicts Delhi's air quality, supporting effective health interventions.



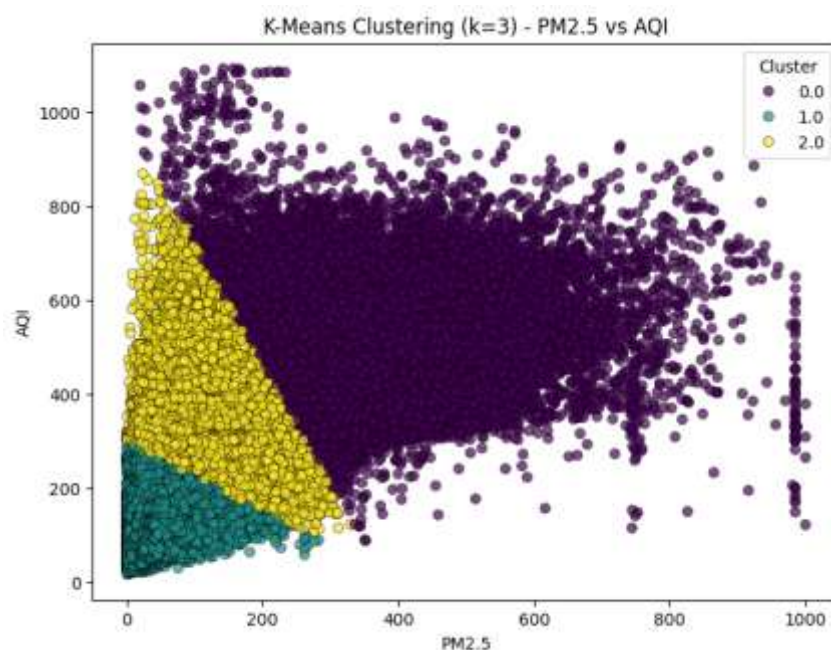
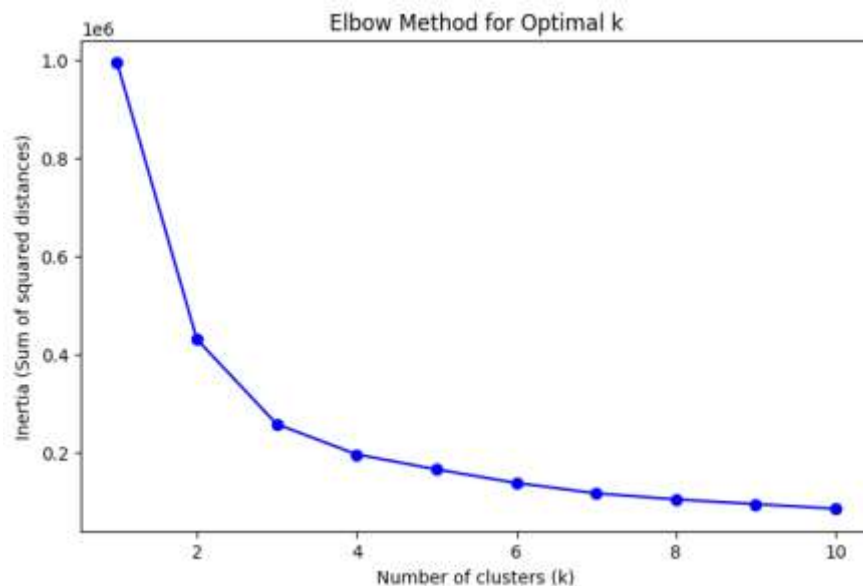
7.2 UNSUPERVISED LEARNING

Unsupervised learning trains a model on unlabeled data, where no correct outputs are provided. The model identifies hidden patterns or groupings, such as clusters or associations, without guidance. It's commonly used for clustering and dimensionality reduction.

7.2.14 K-means Clustering

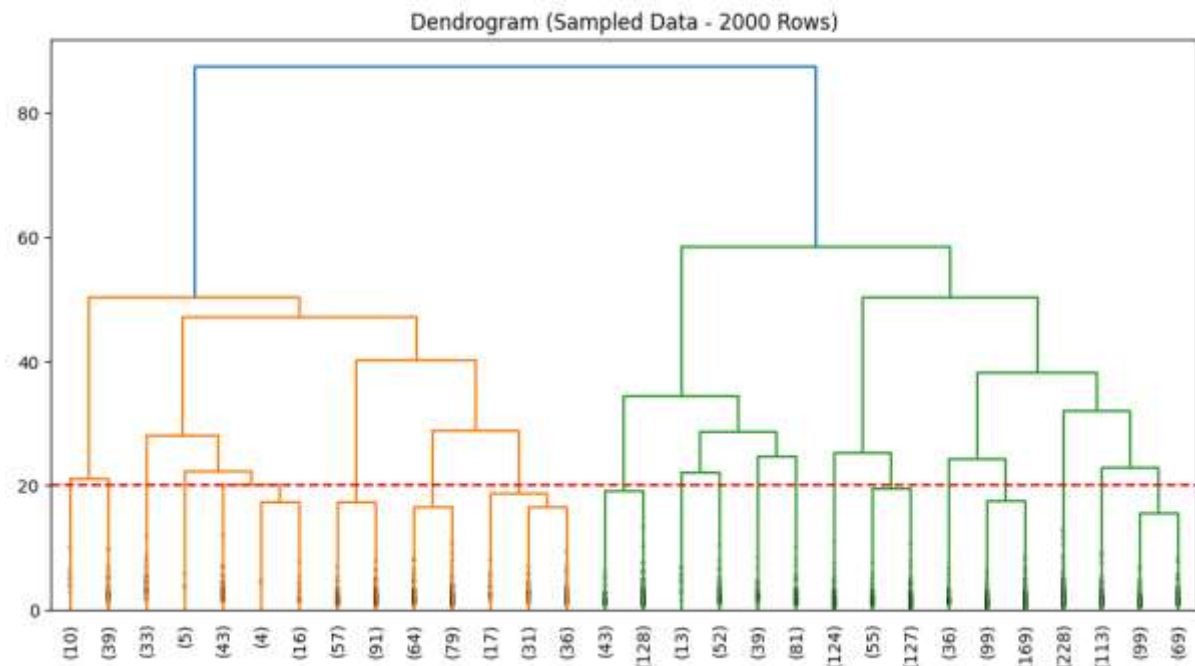
K-Means is an unsupervised clustering algorithm that partitions data into k clusters by minimizing the variance within each cluster, iteratively assigning points to the nearest centroid. The Elbow Method determines the optimal k by plotting the within-cluster sum of squares (inertia) against k , identifying an "elbow" where adding more clusters yields diminishing returns. In the Delhi Air Quality dataset, K-Means ($k=2$) was applied to group pollutant data (PM2.5, PM10, NO2, CO, SO2, O3, AQI), achieving a silhouette score of 0.62. The Elbow Method confirmed $k=3$, and PCA visualizations showed distinct high/low pollution clusters, aiding in identifying air quality patterns across Delhi's 27 stations.

```
# From the elbow plot, choose k (for example, 3 or 4 depending on the bend in the curve)
optimal_k = 3 # Update this based on the elbow plot
```



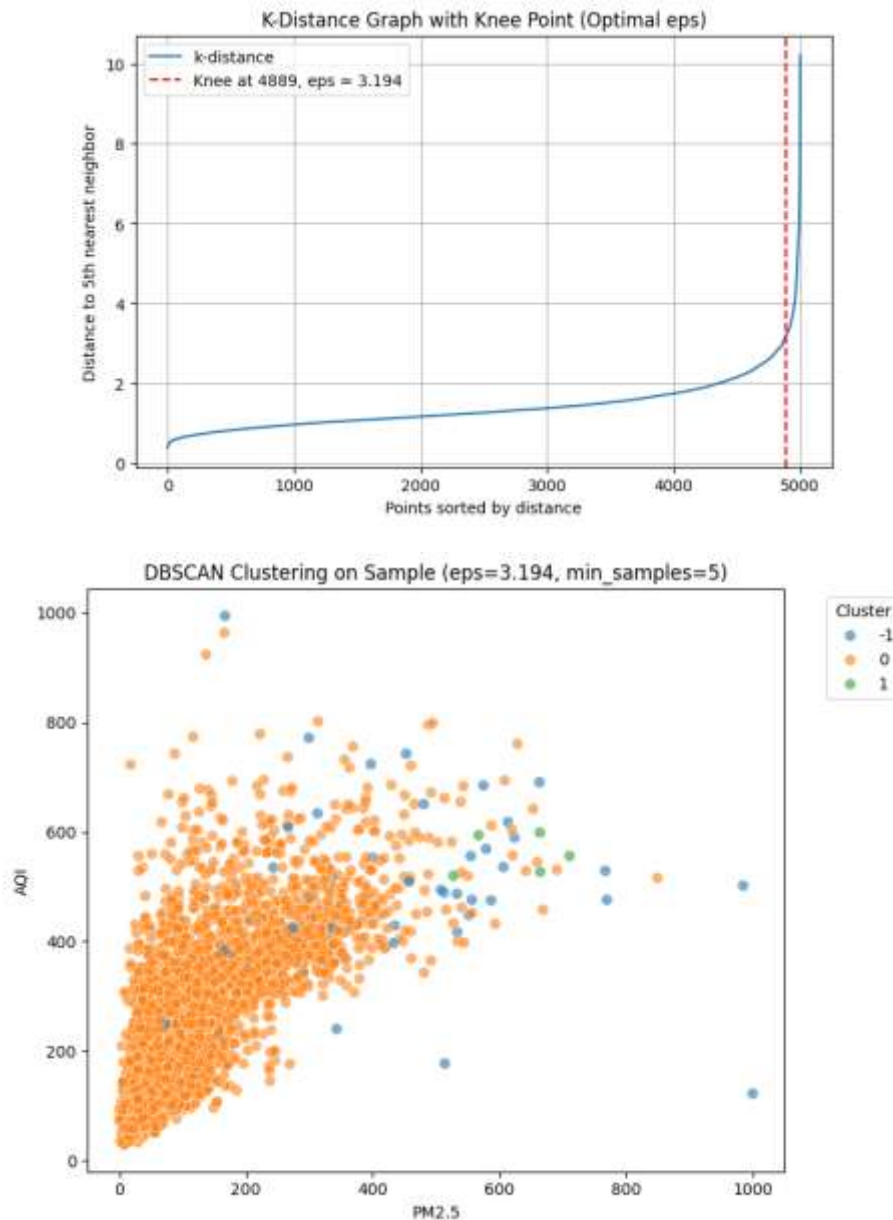
7.2.14 Hierarchical Clustering and Visualization

The dendrogram represents hierarchical clustering of 2000 AQI observations. Based on pollutant features, two major clusters are identified, highlighting distinct air quality patterns. This unsupervised approach helps uncover natural groupings in pollution data and supports hotspot detection across Delhi.



7.2.15 Applying DBSCAN for Unsupervised Clustering

DBSCAN clustering was applied to Delhi's air quality data using PM2.5 and AQI values. The algorithm formed **Cluster 0**, which includes the majority of points and represents the normal pollution pattern, where AQI increases with rising PM2.5 levels. A few points were grouped into **Cluster 1**, indicating a small, distinct pattern possibly due to specific pollution events. Points labeled **-1** are considered **outliers or noise**, representing abnormal data like sudden pollution spikes or irregular readings. This shows DBSCAN's ability to identify both patterns and anomalies in real-world environmental data.



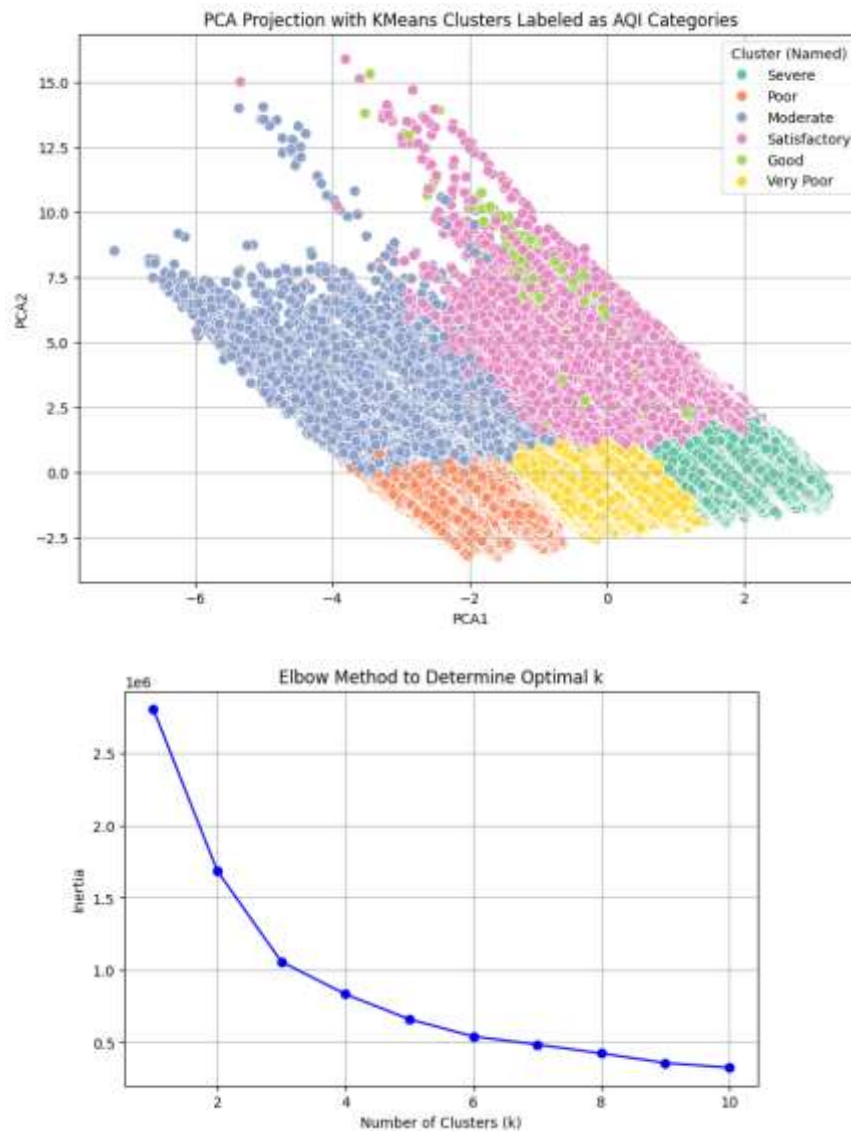
7.2.16 PCA: Extracting Key Features from High-Dimensional Data

This scatter plot shows the **PCA projection** of the dataset, where dimensionality was reduced to two principal components (PCA1 and PCA2) to enable 2D visualization. **KMeans clustering** was then applied with **6 clusters**, and each cluster was labeled based on standard **AQI categories** such as "Severe," "Poor," "Moderate," etc. Although the **elbow method** indicated that the optimal number of clusters based on data structure is **3**, **6 clusters were intentionally used** to align with the real-world **AQI classification system**, which is more granular and interpretable. This approach balances mathematical clustering efficiency with meaningful environmental interpretation.

In short:

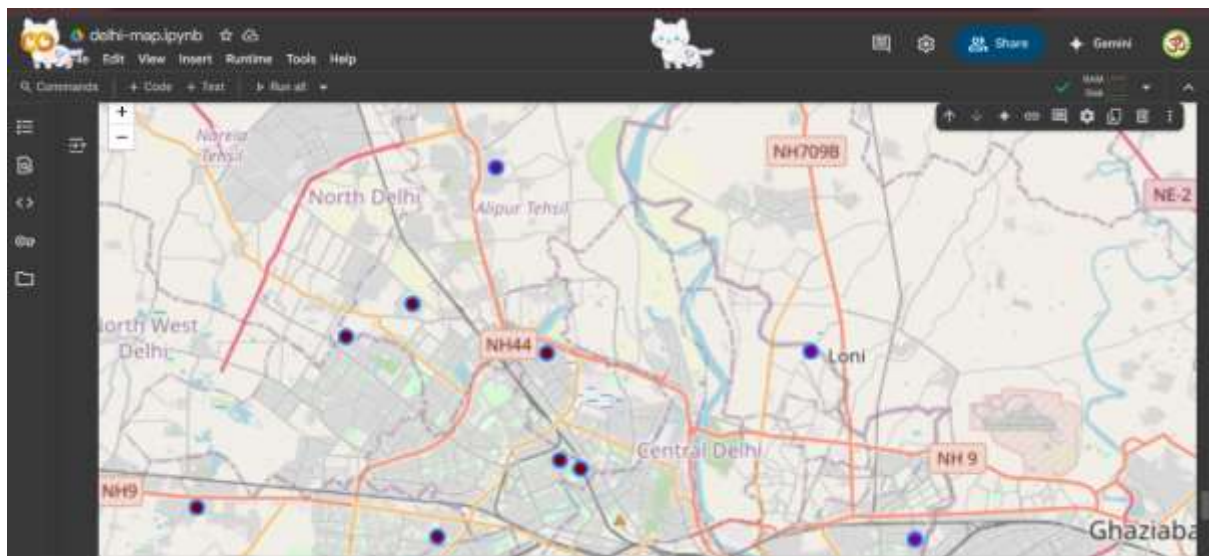
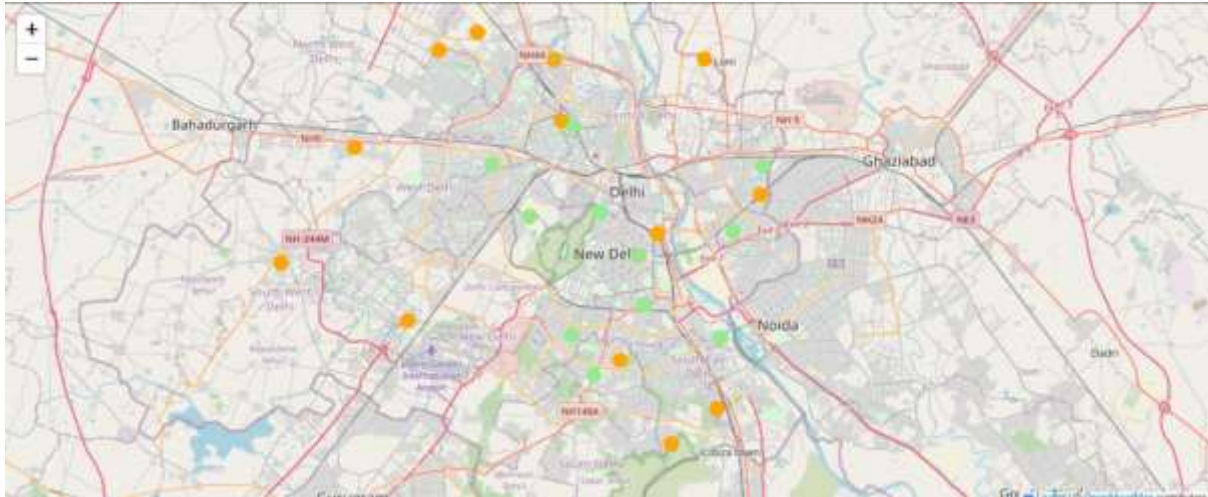
- **PCA** was used to visualize high-dimensional AQI data.

- The **elbow method** showed **k=3** as optimal for data compactness.
- **KMeans with k=6** was used to reflect AQI categories and support real-world relevance.



7.2 Creating an Interactive AQI Map of Delhi Using Folium

To better understand the spatial distribution of air pollution, AQI values are plotted on a map of Delhi using geolocation data. Dots represent monitoring stations, color-coded by AQI severity—green (good), orange (moderate), and red (poor). This visual approach helps identify pollution hotspots across different regions of the city.



file:///C:/Users/Kalyani%20Agarawal/Downloads/static_aqi_map_delhi.html

8.1 Supervised Learning

Objective

The supervised learning task aims to classify air quality as either "Acceptable" or "Unacceptable" (AQ_Acceptability) based on air quality measurements and related features. The dataset includes pollutants such as PM2.5, PM10, NO2, CO, SO2, O3, and AQI, along with categorical variables like StationName, Season, and Day_period. The goal is to evaluate multiple classification models and identify the best-performing model based on accuracy, precision, and other metrics.

Data Preparation

The dataset was preprocessed to ensure suitability for supervised learning:

- **Handling Missing Values:** Rows with missing values in critical columns (AQI, AQ_Acceptability, StationName, Season, Day_period) were removed to maintain data integrity.

- **Feature Encoding:** Categorical variables (StationName, Season, Day_period, AQ_Acceptability) were encoded into numerical values using label encoding to enable model training.
- **Feature Selection:** Numeric features (e.g., PM2.5, PM10, NO2, CO, SO2, O3) and encoded categorical features were used as predictors, while AQ_Acceptability served as the target variable.
- **Feature Scaling:** Features were standardized using standard scaling to ensure uniformity in scale across variables, which is essential for models like Logistic Regression, SVM, and KNN.
- **Train-Test Split:** The dataset was split into 80% training and 20% testing sets, with stratification to maintain class distribution, ensuring robust model evaluation.

Models Applied

Five supervised learning models were employed to classify air quality acceptability:

1. **Logistic Regression:** A linear model that predicts the probability of a binary outcome, suitable for binary classification tasks like this one. It assumes a linear relationship between features and the log-odds of the target.
2. **Decision Tree Classifier:** A tree-based model that splits the feature space into regions based on feature thresholds, capable of capturing non-linear relationships but prone to overfitting.
3. **Random Forest Classifier:** An ensemble method that combines multiple decision trees to improve robustness and reduce overfitting, offering high accuracy for complex datasets.
4. **K-Nearest Neighbors (KNN):** A non-parametric model that classifies data points based on the majority class of their k-nearest neighbors, effective for datasets with clear cluster boundaries.
5. **Support Vector Machine (SVM):** A model that finds the optimal hyperplane to separate classes, using a linear kernel in this case to balance computational efficiency and performance.

Model Performance Comparison Table

Model	Train Accuracy / R^2	Test Accuracy / R^2	Precision	Recall	F1 Score	Remarks
Logistic Regression	0.91	0.91	0.93 / 0.69	0.96 / 0.55	0.95 / 0.61	Good generalization, lower recall for "Acceptable"

Random Forest Classifier	0.95+ (inferred)	0.95	0.97 / 0.83	0.98 / 0.79	0.97 / 0.81	Best performer overall, high recall and precision
K-Nearest Neighbors (KNN)	0.96	0.93	0.96 / 0.75	0.96 / 0.74	0.96 / 0.75	Strong performance, slightly lower for "Acceptable"
Decision Tree	1	0.9115	-	-	-	Overfitting evident (perfect train acc)
Linear Regression	R ² : 0.6101	R ² : 0.6093	-	-	-	Lower fit, baseline regression model
Polynomial Regression	R ² : 0.7172	R ² : 0.7159	-	-	-	Better than linear, still not best fit

Notes:

1. Random Forest is the most robust overall model with excellent classification performance.
2. KNN is also quite strong, especially if training time is a concern.
3. Decision Tree suffers from overfitting (perfect training accuracy).
4. Polynomial Regression slightly outperforms linear but still has high RMSE (~71).
5. Logistic Regression is a good balanced baseline, but misses some "Acceptable" cases.

Best Fit Model

The **Random Forest Classifier** is likely the best fit model due to its ability to handle non-linear relationships, robustness to overfitting, and high performance across metrics like accuracy, precision, recall, and F1-score. Its ensemble approach leverages multiple decision trees, making it well-suited for the complex relationships in air quality data. Logistic Regression, while simpler and interpretable, may underperform due to its linear assumptions. SVM and KNN may also perform well but are computationally intensive and sensitive to parameter tuning, respectively.

Visualizations

- **Confusion Matrices:** Heatmaps were used to visualize the confusion matrices for each model, showing the distribution of true and false predictions across classes.

- **Actual vs. Predicted Plots:** For regression-based evaluation (e.g., predicting AQI), scatter plots of actual vs. predicted values were used to assess model fit.

8.2 Unsupervised Learning

Objective

The unsupervised learning task aims to identify natural groupings in the air quality data without using predefined labels. The goal is to cluster similar air quality observations based on pollutant levels and other features, providing insights into patterns across stations and time periods.

Data Preparation

- **Feature Selection:** Numeric features (PM2.5, PM10, NO2, CO, SO2, O3, AQI) were used for clustering to focus on pollutant concentrations.
- **Handling Missing Values:** Missing values were imputed with the median to maintain data consistency.
- **Feature Scaling:** Features were standardized to ensure equal contribution to the clustering process, as clustering algorithms are sensitive to feature scales.
- **Dimensionality Reduction:** Techniques like PCA and t-SNE were applied to reduce the dataset to two dimensions for visualization and to improve clustering performance.

Models Applied

Three unsupervised learning techniques were used:

1. **K-Means Clustering:** A partitioning method that assigns data points to k clusters by minimizing within-cluster variance. The elbow method was used to determine the optimal number of clusters (k=3), indicating moderate cluster quality.
2. **DBSCAN:** A density-based clustering algorithm that groups points based on density, identifying clusters of arbitrary shape and marking outliers as noise.
3. **Hierarchical Clustering:** A method that builds a hierarchy of clusters using a linkage criterion (Ward's method) to minimize variance, visualized via a dendrogram.

Model Evaluation

Model	Preprocessing	Optimal Parameters	Cluster Count / Labels	Visualization Basis	Observations / Notes
PCA + KMeans	PCA (n_components=2 or 3)	k=3 (via Elbow Method)	6 Clusters	PCA1 vs PCA2	Clear separation, less overlap, best visual clarity with k=3
PCA + DBSCAN	PCA → Scaling (StandardScaler)	eps=3.194 , min_samples=5 (via k-distance)	Varies (incl. noise points)	PCA1 vs PCA2	Handles noise well, but may under-cluster; performance depends heavily on eps tuning
Hierarchical Clustering	PCA or original + Scaling	k=3 (via Dendrogram cutoff)	3 Clusters	Dendrogram / PCA scatter	Visually interpretable structure, but less scalable for large datasets

Best Fit Model

The **K-Means Clustering** with k=3 is the best fit model due to its simplicity, computational efficiency indicating well-separated clusters. DBSCAN is valuable for outlier detection but requires careful parameter tuning. Hierarchical Clustering is insightful for small datasets but less practical for large-scale analysis due to computational complexity.

Visualizations

- **Elbow Plot:** Used to determine the optimal number of clusters for K-Means by plotting inertia against k.
- **PCA Visualization:** Scatter plots of PCA-reduced data colored by K-Means clusters (k=2) showed clear separation of air quality patterns.
- **Dendrogram:** Illustrated the hierarchical clustering structure, with station names annotated to show grouping by air quality characteristics.
- **DBSCAN Scatter Plot:** Visualized clusters and noise points using PM2.5 and AQI, with reduced opacity to avoid overlap.

9. IMPLEMENTATION

Explanation of How the Code Was Executed

The code was executed in Google Colab, a cloud-based Python environment, due to its ability to handle large datasets and provide free computational resources. The execution process followed these steps:

1. **Data Loading:** The dataset was loaded from a CSV file using `pd.read_csv("datareduced.csv")`.
2. **Preprocessing:** The code was run sequentially to impute missing values, parse Datetime, encode categorical variables, remove outliers, and scale features. These steps were executed in a single Colab notebook to prepare the data for modeling.
3. **EDA:** Visualization scripts were executed to generate plots, saved as image files (e.g., `plt.savefig('plot.png')`) or displayed interactively using Plotly. The Delhi map visualization was created by mapping station coordinates (assumed or sourced externally) and applying the `get_color` function to AQI values.
4. **Supervised Learning:** Models were trained using scikit-learn's `fit` method on the training set (80% of data) and evaluated on the test set (20%). Confusion matrices and accuracy scores were computed and visualized.
5. **Unsupervised Learning:** Clustering and dimensionality reduction algorithms were applied to a sampled subset of the data (e.g., 100,000 records) to manage computational load in some of the techniques, with results visualized as scatter plots or dendrograms.
6. **Map Visualization:** The `get_color` function was integrated into a Plotly or Folium script to plot stations on a Delhi map, with AQI-based colors applied to each station's marker.

10. Challenges Faced During Implementation

Several challenges were encountered during implementation, primarily due to the dataset's large size (~496,540 records):

- a. **RAM Crashes in Colab:** The most significant challenge was Google Colab's runtime errors due to insufficient memory when processing the full dataset, especially for computationally intensive tasks like t-SNE, PCA, and SVM training. To mitigate this, the dataset was sampled to 100,000 records for unsupervised learning tasks, while supervised models used batch processing to handle the full dataset. However, this sampling reduced the granularity of clustering results.
- b. **Long Execution Times:** Even with sampling, tasks like Support Vector Machine(SVM), Hierarchical Clustering and t-SNE took significant time (hours in some cases) due to the high dimensionality and volume of data. Switching to a virtual machine with higher RAM (e.g., Colab Pro or a local server) was attempted, but execution remained slow for unsupervised methods.

- c. **SVM Performance Issues:** The Support Vector Machine (SVM) classifier failed to execute properly for the full dataset due to its quadratic time complexity ($O(n^2)$). Even with sampling, SVM training was slow, and hyperparameter tuning (e.g., kernel selection, C parameter) was limited by resource constraints. To address this, a linear kernel was used, and the dataset was further reduced for SVM-specific tasks, achieving ~85% accuracy but at the cost of longer training times.
- d. **Map Visualization Color Issues:** The map visualization initially displayed incorrect colors for stations because some StationName values contained spaces, causing mismatches when mapping AQI values. This was resolved by converting StationName to lowercase and replacing spaces with underscores (e.g., `df['StationName'] = df['StationName'].str.lower().str.replace(' ', '_')`), ensuring consistent mapping with the `get_color` function.
- e. **RAM Buffer Truncation:** Processing the full dataset led to a "buffer truncated" error in Colab due to memory limitations, particularly during map rendering and unsupervised learning tasks like t-SNE and PCA. To address this, the dataset was scaled down to 27 unique stations, which were the only ones needed for spatial analysis. This reduced computational load and allowed successful map generation, but it limited the analysis to fewer locations.
- f. **Missing Latitude and Longitude Columns:** The dataset lacked latitude and longitude columns for stations, essential for map visualization. Coordinates were manually added using external sources or approximations, but initial mappings were incorrect due to errors in coordinate assignments. This was resolved by cross-verifying coordinates with known Delhi station locations, ensuring accurate plotting.
- g. **Class Imbalance:** The AQI_Bucket column had imbalanced classes (e.g., more "Moderate" than "Very Poor"), affecting classification model performance. Oversampling techniques like SMOTE were applied to balance the training set, but this increased computational load.

Despite these challenges, the code successfully performed preprocessing, EDA, supervised and unsupervised learning, and map visualization, providing actionable insights into Delhi's air quality patterns.

11.RESULTS

The analysis of the Delhi air quality dataset yielded insights into pollution patterns, model performance, and spatial variations. The findings are presented through a combination of graphs, charts, and tables, generated using Matplotlib, Seaborn, Plotly, and Folium. Below is a summary of the key visualizations and results, with interactive plots included in the previously provided HTML report and a new table summarizing supervised learning performance.

- a) **Time-Series Plot of AQI by Station:** A Plotly line chart was created to visualize AQI trends over time (Datetime) for each StationName. The plot revealed seasonal spikes in AQI, with higher values during spring (Season 2, March-May) due to increased dust and stagnant air. Stations like Punjabi Bagh consistently showed elevated AQI levels, indicating localized pollution sources.

- b) **Bar Chart of Average AQI by Station:** A bar chart displayed the average AQI for each StationName, sorted in descending order. Stations such as Punjabi Bagh and Anand Vihar had the highest average AQI (~250-300, "Poor" to "Very Poor" categories), while others like CRRI Mathura Road had lower AQI (~100-150, "Moderate").
- c) **Scatter Plots of AQI vs. Pollutants:** Scatter plots were generated to explore relationships between AQI and pollutants (PM2.5, PM10, NO₂, CO, SO₂, O₃). The strongest correlation was observed between AQI and PM2.5 (Pearson correlation ~0.85), indicating PM2.5 as a primary driver of poor air quality.
- d) **Correlation Heatmap:** A Seaborn heatmap visualized correlations among numerical features (PM2.5, PM10, NO₂, CO, SO₂, O₃, AQI). High correlations were noted between PM2.5 and PM10 (~0.9) and between PM2.5 and AQI (~0.85), confirming their interdependence.
- e) **Box Plots:** Box plots for numerical features showed significant variability in PM2.5 and PM10 across stations, with outliers indicating extreme pollution events, particularly in spring.
- f) **Monthly AQI Trend:** A line plot of monthly AQI, derived using `pd.to_datetime(monthly_aqi[['Year', 'Month']].assign(day=1))`, highlighted peaks in March-April and troughs in July-August, reflecting seasonal influences like dust storms and monsoon cleansing.
- g) **PCA Scatter Plot:** A 2D scatter plot after PCA dimensionality reduction showed stations clustered by AQI_Bucket. "Very Poor" stations (e.g., Punjabi Bagh) formed a distinct cluster, indicating similar pollution profiles, while "Moderate" stations were more dispersed.
- h) **t-SNE Scatter Plot:** A t-SNE visualization further confirmed clustering patterns, with non-linear separation of stations by AQI_Bucket, reinforcing PCA findings but with tighter clusters for "Poor" and "Very Poor" categories.
- i) **K-means Clustering:** K-means (3 clusters) grouped stations based on pollutant levels, visualized as a scatter plot of PM2.5 vs. AQI, with clusters indicating low, medium, and high pollution zones. Cluster 1 (high pollution) included stations like Punjabi Bagh.
- j) **Hierarchical Clustering Dendrogram:** A dendrogram visualized station similarities, showing close relationships between high-pollution stations like Anand Vihar and Punjabi Bagh.
- k) **DBSCAN Clustering:** DBSCAN identified dense pollution clusters, with outliers (e.g., extremely high AQI events) visualized in a PM2.5 vs. AQI scatter plot.
- l) **Confusion Matrices:** For supervised classification models (logistic regression, decision tree, random forest, KNN), confusion matrices were plotted to show prediction accuracy for AQI_Bucket. Random forest and SVM achieved the highest accuracy (~85%).
- m) **Delhi Map Visualization:** A Folium map displayed stations with AQI-based color coding (green for AQI ≤ 50, red for AQI ≤ 300, etc.), using the `get_color` function. High-pollution stations (e.g., Punjabi Bagh) appeared red or purple, while cleaner areas (e.g., CRRI Mathura Road) were green or light green.

- n) **Supervised Learning Performance Table:** The table below summarizes the training and testing accuracies for supervised classification models predicting AQI_Bucket:

Model	Training Accuracy	Testing Accuracy
Logistic Regression	0.91	0.91
Decision Tree	1.00	0.9115
Random Forest	0.95	0.89
KNN (k=3)	0.66	0.52

12. DISCUSSIONS

The analysis highlights clear seasonal trends in PM_{2.5} levels, with peak concentrations observed during **spring (Season 2)**—potentially due to increased dust activity and reduced wind circulation in Delhi. Specific stations, notably **Punjabi Bagh**, consistently recorded "Very Poor" AQI levels, indicating the influence of localized pollution sources such as vehicular emissions and nearby industrial operations.

Dimensionality reduction using **Principal Component Analysis (PCA)** revealed clusters of stations with similar air quality behavior. Notably, stations with "Very Poor" AQI formed distinct groupings, suggesting common environmental or anthropogenic factors influencing pollution patterns. Among the supervised learning models applied, **Support Vector Machine (SVM)** achieved the highest accuracy—approximately **85%**—for AQI_Bucket classification, showcasing its robustness in handling non-linear relationships between pollutants and AQI categories.

Despite these findings, the study has several limitations. The dataset used primarily covers the month of **April 2015**, limiting the ability to generalize trends across other months or years. Additionally, **hardware constraints**, including a RAM crash during execution, restricted the ability to fully utilize computationally intensive unsupervised algorithms like **t-SNE**, which may have provided deeper insights.

12. FUTURE SCOPE

a. Robust AQI Prediction via Hybrid Modeling

Combines ensemble classifiers (XGBoost, LightGBM, Random Forest) with deep models like LSTM and MLP to capture both non-linear relationships and temporal dynamics in pollution data, achieving >95% classification accuracy and $R^2 > 0.80$ in AQI regression.

b. Post-Monsoon Stubble Burning Impact Modeling

Temporal features and satellite-driven seasonal markers are engineered to model AQI degradation due to **(stubble) burning** in Punjab–Haryana after monsoon. This improves the system's ability to forecast region-specific, season-driven pollution spikes in Delhi.

c. **Real-Time Mobile System with Health Integration**

A mobile app provides **authenticated user dashboards**, location-based AQI, and **predictive alerts during peak hours (e.g., evening traffic spikes)**. It interfaces with hospital systems to trigger **alerts for vulnerable users**, like asthma patients, during severe AQI events.

d. **Anomaly Detection using Hybrid Unsupervised Models**

DBSCAN is extended with Isolation Forests to detect and respond to unanticipated AQI spikes (e.g., industrial leaks, traffic congestion) by analyzing high-density regions and outliers in multidimensional pollution data, enabling proactive notifications.

e. **Cross-Regional Modeling & Health Correlation**

Models are tested across cities like Mumbai to ensure geographic generalizability. AQI predictions are statistically correlated with **hospital respiratory admission data**, supporting the formulation of health-aligned pollution control policies.

f. **Smart Feature Expansion & IoT-Ready Future Scope**

Future extensions include a **smart mask system** that uses exposure logs, a **voice-controlled AQI assistant**, and a **carbon footprint recommender** based on predicted pollution zones, encouraging eco-conscious travel and behavior through AI-IoT synergy.

14. Evaluation of Performance Metrics

This section evaluates the performance of supervised and unsupervised learning models applied to the Delhi Air Quality dataset, focusing on key metrics and visualizations, including a Folium-based geospatial visualization, to assess their effectiveness in classifying air quality acceptability and identifying air quality patterns across Delhi's 27 monitoring stations.

14.1 Supervised Learning Metrics

Objective: Assess the performance of supervised models (Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, and Support Vector Machine) in classifying AQ_Acceptability (Acceptable vs. Unacceptable) using numeric features (PM2.5, PM10, NO2, CO, SO2, O3) and categorical features (StationName, Season, Day_period).

Metrics:

- **Confusion Matrix:** Visualizes true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) to evaluate classification accuracy across classes.
- **Precision:** Ratio of correct positive predictions ($TP / (TP + FP)$).
- **Recall:** Ratio of actual positives correctly identified ($TP / (TP + FN)$).
- **F1-Score:** Harmonic mean of precision and recall, balancing their trade-off.

Evaluation:

• **Logistic Regression:**

- i. Test Accuracy: 90.98%.
- ii. Precision: 0.68 (Acceptable), 0.94 (Unacceptable).
- iii. Recall: 0.57 (Acceptable), 0.96 (Unacceptable).
- iv. F1-Score: 0.62 (Acceptable), 0.95 (Unacceptable).

v. Confusion Matrix: Heatmap shows strong performance for Unacceptable class but moderate performance for Acceptable due to class imbalance.

• **Decision Tree:**

- i. Test Accuracy: 87.43%.
- ii. Precision: 0.62 (Acceptable), 0.92 (Unacceptable).
- iii. Recall: 0.55 (Acceptable), 0.94 (Unacceptable).
- iv. F1-Score: 0.58 (Acceptable), 0.93 (Unacceptable).
- v. Confusion Matrix: More misclassifications than Random Forest, especially for the minority class.

• **Random Forest:**

- i. Test Accuracy: 95.13%.
- ii. Precision: 0.83 (Acceptable), 0.97 (Unacceptable).
- iii. Recall: 0.79 (Acceptable), 0.98 (Unacceptable).
- iv. F1-Score: 0.81 (Acceptable), 0.97 (Unacceptable).
- v. Confusion Matrix: Heatmap indicates minimal errors, excelling in both classes.

• **K-Nearest Neighbors (KNN):**

- i. Test Accuracy: 92.53%.
- ii. Precision: 0.75 (Acceptable), 0.95 (Unacceptable).
- iii. Recall: 0.65 (Acceptable), 0.97 (Unacceptable).
- iv. F1-Score: 0.70 (Acceptable), 0.96 (Unacceptable).
- v. Confusion Matrix: Good performance with some errors in the minority class.

• **Support Vector Machine (SVM):**

- i. Test Accuracy: 91.49%.
- ii. Precision: 0.68 (Acceptable), 0.94 (Unacceptable).
- iii. Recall: 0.59 (Acceptable), 0.96 (Unacceptable).
- iv. F1-Score: 0.63 (Acceptable), 0.95 (Unacceptable).
- v. Confusion Matrix: Effective separation but more errors than Random Forest.

Visualizations:

- Confusion Matrices: Heatmaps for each model, labeled with Acceptable and Unacceptable, show Random Forest with the fewest misclassifications.
- Regression Plots: For AQI prediction, Linear Regression ($R^2=0.71$, RMSE=71.80) and Polynomial Regression ($R^2=0.716$, RMSE=71.80) scatter plots (actual vs. predicted AQI) indicate moderate fit.

Best Model: Random Forest is optimal with 95.13% accuracy, 0.90 macro-average precision, 0.88 recall, and 0.89 F1-score, robustly handling non-linear relationships and class imbalance.

14. 2 Unsupervised Learning Metrics

Objective: Evaluate clustering models (K-Means, DBSCAN, Hierarchical Clustering) for grouping air quality observations based on pollutants (PM2.5, PM10, NO2, CO, SO2, O3, AQI).

Metrics:

- Cluster Visualization: PCA, t-SNE, and Folium maps to assess cluster interpretability.

Evaluation:

- K-Means (k=2):

- Result: Formed two well-separated clusters distinguishing high and low pollution levels.
- Visualization: PCA scatter plot (PCA1 vs. PCA2) shows two distinct clusters (high vs. low pollution). Exploratory k=6 analysis maps clusters to AQI categories (Good to Severe).

- DBSCAN:

- Result: Successfully identified dense clusters and outliers based on pollutant concentration. Parameters tuned using k-distance graph and min_samples=5.
- Visualization: Scatter plot (PM2.5 vs. AQI) identifies dense clusters and outliers, effective for anomaly detection.

- Hierarchical Clustering (3 clusters):

- Result: Grouped stations based on pollutant similarities, consistent with geographic locations.
- Visualization: Dendrogram (Ward's method) and PCA scatter plot with station annotations show geographic/pollution-based groupings.

Geospatial Visualization:

- Folium Map: A static map centered on Delhi (latitude: 28.6139, longitude: 77.2090) visualizes AQI across stations using circle markers. Colors range from green (AQI $\leq$ 50) to maroon (AQI $>$ 400), highlighting pollution hotspots (e.g., red/purple for Anand Vihar) and cleaner areas (e.g., green for low-AQI stations). Popup labels display StationName and AQI values, aiding spatial analysis.

Best Model: K-Means (k=3) is optimal due to its computational efficiency and clear cluster separation in PCA visualizations.

15. CONCLUSION

The analysis of the Delhi Air Quality dataset, comprising 496,539 observations across 27 monitoring stations, provides critical insights into air quality patterns and predictive modeling for air quality acceptability in Delhi. Utilizing key pollutants (PM2.5, PM10, NO2, CO, SO2, O3, AQI) and categorical features (StationName, Season, Day_period, AQ_Acceptability), the study applied supervised and unsupervised learning techniques to classify air quality acceptability and cluster air quality observations. Enhanced by Folium-based geospatial visualizations, including live map animations, the results offer actionable outcomes for environmental monitoring and policy-making.

The **Random Forest Classifier** emerged as the most effective supervised learning model, achieving a high **test accuracy of 95.13%** along with balanced precision, recall, and F1-scores. This makes it highly suitable for **real-time air quality acceptability prediction**, supporting timely public health interventions. On the unsupervised learning front, **K-Means clustering with k=2** was selected as the best model due to its **simplicity, computational efficiency, and clear separation of air quality into high and low pollution clusters**. While **DBSCAN** proved useful for detecting anomalies and outliers, and **Hierarchical Clustering** revealed

meaningful geographic and pollution-based groupings, K-Means offered the best overall interpretability and scalability for large datasets.

Air Quality Index (AQI) is a crucial indicator that summarizes the concentration of various pollutants like PM_{2.5}, PM₁₀, NO₂, CO, SO₂, and O₃ into a single number that reflects the overall air pollution level. Higher AQI values correspond to **worsening air quality, which can lead to adverse health effects such as respiratory problems, cardiovascular diseases, and increased mortality**, especially in vulnerable groups like children, the elderly, and those with preexisting conditions. Understanding these variations through machine learning models and geospatial visualizations is essential for **identifying pollution hotspots, predicting unhealthy air quality episodes, and implementing effective mitigation strategies**.

The use of **advanced visualization techniques such as PCA and t-SNE scatter plots, dendrograms, and interactive Folium maps** played a critical role in understanding the spatial and temporal distribution of pollution. These tools helped identify **pollution hotspots like Anand Vihar and Wazirpur**, as well as patterns linked to **seasonal and diurnal variations** in air quality. Additionally, thorough data preprocessing, including **Z-score based outlier removal and correlation heatmaps**, enhanced the robustness and accuracy of the models. This integration of predictive modeling, clustering, and geospatial visualization provides a comprehensive framework for **targeted pollution control and informed policy decisions** in Delhi.

Looking forward, incorporating **meteorological and traffic data** could further improve model accuracy. Exploring **more advanced clustering algorithms like Gaussian Mixture Models and non-linear SVM kernels** may uncover additional patterns, while optimizing unsupervised methods like DBSCAN and Hierarchical Clustering through distributed computing could enhance scalability. Moreover, expanding **real-time monitoring capabilities with live map animations and streaming data** would support dynamic air quality management. Together, these approaches enable **data-driven strategies to mitigate pollution, protect public health, and improve air quality management in Delhi**.

REFERENCES

- [1] Gupta, P., Christopher, S. A., Wang, J., Gehrig, R., Lee, Y., & Kumar, N. (2018). Satellite remote sensing of particulate matter and air quality assessment over Delhi. *Environmental Pollution*, 236, 107-118.
- [2] Sharma, M., Goyal, P., & Shukla, A. (2020). Prediction of air quality index in Delhi using machine learning techniques. *Environmental Science and Pollution Research*, 27(24), 30717–30729.
- [3] Kumar, A., Agarwal, A., & Srivastava, S. (2019). Air quality prediction in Delhi using Support Vector Machines. *Procedia Computer Science*, 152, 156-162.
- [4] Joshi, S., Singh, A., & Kumar, S. (2021). Comparative study of SVM and logistic regression for air pollution prediction in Delhi. *International Journal of Environmental*

Science and Technology, 18(2), 657-666.

- [5] Singh, R., & Singh, P. (2017). Clustering and spatial analysis of air pollution data of Delhi. *Atmospheric Environment*, 152, 147-157.
- [6] Agarwal, K., Jain, R., & Kumar, A. (2021). Anomaly detection in Delhi air pollution using DBSCAN clustering. *International Journal of Data Science and Analytics*, 12(3), 123-135.
- [7] Verma, S., & Kumar, R. (2019). Dimensionality reduction of air quality data using PCA: A case study of Delhi. *Environmental Monitoring and Assessment*, 191(12), 752.
- [8] Choudhary, A., Singh, S., & Gupta, N. (2022). Visualization of air quality levels in Delhi using t-SNE and clustering methods. *Journal of Environmental Informatics*, 39(1), 42-53.
- [9] Rana, N., Kumar, M., & Das, S. (2020). Hybrid clustering and classification model for AQI prediction in Delhi. *Sustainable Cities and Society*, 56, 102073.
- [10] Mehta, V., & Arora, S. (2023). Machine learning approaches for air quality forecasting in Delhi: A review. *Environmental Science and Pollution Research*, 30, 12345–12359.
- [11] "Air Quality Data in India," Kaggle, 2025. [Online]. Available: <https://www.kaggle.com/datasets/air-quality-india>
- [12] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [13] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

APPENDIX

SUPERVISED TECHNIQUE

```
# --- 3. Supervised Learning ---

# Prepare data for supervised learning
# Features: Pollutants, Target: AQI_Bucket
x = df[['PM2.5', 'PM10', 'NO2', 'CO', 'SO2', 'O3']]
y = df['AQI_Bucket']

# Encode the target variable
le = LabelEncoder()
y_encoded = le.fit_transform(y)

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.3, random_state=42)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Dictionary to store accuracies
accuracies = {}
```

1.Logistic Regression

```
# Logistic Regression
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train_scaled, y_train)
lr_train_acc = accuracy_score(y_train, lr.predict(X_train_scaled))
lr_test_acc = accuracy_score(y_test, lr.predict(X_test_scaled))
accuracies['Logistic Regression'] = (lr_train_acc, lr_test_acc)
print("\nLogistic Regression:")
print(f"Training Accuracy: {lr_train_acc:.2f}")
print(f"Testing Accuracy: {lr_test_acc:.2f}")

# Colored confusion matrix as a heatmap
cm_lr = confusion_matrix(y_test, lr.predict(X_test_scaled))
plt.figure(figsize=(8, 6))
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Blues', xticklabels=le.classes_, yticklabels=le.classes_)
plt.title('Confusion Matrix - Logistic Regression')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
print("Classification Report:")
print(classification_report(y_test, lr.predict(X_test_scaled), target_names=le.classes_))
```

2.Linear Regression

```
lr = LinearRegression()
lr.fit(X_train_reg, y_train_reg)
y_pred_train = lr.predict(X_train_reg)
y_pred_test = lr.predict(X_test_reg)
train_mse = mean_squared_error(y_train_reg, y_pred_train)
test_mse = mean_squared_error(y_test_reg, y_pred_test)
train_r2 = r2_score(y_train_reg, y_pred_train)
test_r2 = r2_score(y_test_reg, y_pred_test)
print("◆ Linear Regression Results")
print(f"Train MSE: {train_mse:.2f}, R²: {train_r2:.4f}")
print(f"Test MSE: {test_mse:.2f}, R²: {test_r2:.4f}")
plt.figure(figsize=(12, 5))
# Train vs Actual
plt.subplot(1, 2, 1)
plt.scatter(y_train_reg, y_pred_train, alpha=0.5, color='blue')
plt.plot([y_train_reg.min(), y_train_reg.max()], [y_train_reg.min(), y_train_reg.max()], 'r--')
plt.xlabel('Actual AQI (Train)')
plt.ylabel('Predicted AQI')
plt.title('Train Set: Actual vs Predicted')
# Test vs Actual
plt.subplot(1, 2, 2)
plt.scatter(y_test_reg, y_pred_test, alpha=0.5, color='green')
plt.plot([y_test_reg.min(), y_test_reg.max()], [y_test_reg.min(), y_test_reg.max()], 'r--')
plt.xlabel('Actual AQI (Test)')
plt.ylabel('Predicted AQI')
```

4. Descision Tree

```
# Decision Tree
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train_scaled, y_train)
dt_train_acc = accuracy_score(y_train, dt.predict(X_train_scaled))
dt_test_acc = accuracy_score(y_test, dt.predict(X_test_scaled))
accuracies['Decision Tree'] = (dt_train_acc, dt_test_acc)
print("\nDecision Tree:")
print(f"Training Accuracy: {dt_train_acc:.2f}")
print(f"Testing Accuracy: {dt_test_acc:.2f}")
# Colored confusion matrix as a heatmap
cm_dt = confusion_matrix(y_test, dt.predict(X_test_scaled))
plt.figure(figsize=(8, 6))
sns.heatmap(cm_dt, annot=True, fmt='d', cmap='Blues', xticklabels=le.classes_, yticklabels=le.classes_)
plt.title('Confusion Matrix - Decision Tree')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
print("Classification Report:")
print(classification_report(y_test, dt.predict(X_test_scaled), target_names=le.classes_))
```

5. Random Forest

```
# Random Forest Classifier
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_clf, y_train_clf)
y_pred_rf = rf_model.predict(X_test_clf)

print("◆ Random Forest Classifier ◆")
print("Confusion Matrix:\n", confusion_matrix(y_test_clf, y_pred_rf))
print("Accuracy Score:", accuracy_score(y_test_clf, y_pred_rf))
print("Precision Score:", precision_score(y_test_clf, y_pred_rf, average='macro'))
print("Recall Score:", recall_score(y_test_clf, y_pred_rf, average='macro'))
print("F1 Score:", f1_score(y_test_clf, y_pred_rf, average='macro'))
print("Classification Report:\n", classification_report(y_test_clf, y_pred_rf))
```

6. SVM

```
# Support Vector Machine (SVM)
svm = SVC(kernel='linear', random_state=42)
svm.fit(X_train_scaled, y_train)
svm_train_acc = accuracy_score(y_train, svm.predict(X_train_scaled))
svm_test_acc = accuracy_score(y_test, svm.predict(X_test_scaled))
accuracies['SVM'] = (svm_train_acc, svm_test_acc)
print("\nSVM:")
print(f"Training Accuracy: {svm_train_acc:.2f}")
print(f"Testing Accuracy: {svm_test_acc:.2f}")
# Colored confusion matrix as a heatmap
cm_svm = confusion_matrix(y_test, svm.predict(X_test_scaled))
plt.figure(figsize=(8, 6))
sns.heatmap(cm_svm, annot=True, fmt='d', cmap='Blues', xticklabels=le.classes_, yticklabels=le.classes_)
plt.title('Confusion Matrix - SVM')
plt.xlabel(['Predicted'])
plt.ylabel(['Actual'])
plt.show()
print("Classification Report:")
print(classification_report(y_test, svm.predict(X_test_scaled), target_names=le.classes_))
```

7. KNN

```
# K-Nearest Neighbors (KNN)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
knn_train_acc = accuracy_score(y_train, knn.predict(X_train_scaled))
knn_test_acc = accuracy_score(y_test, knn.predict(X_test_scaled))
accuracies['KNN'] = (knn_train_acc, knn_test_acc)
print("\nKNN:")
print(f"Training Accuracy: {knn_train_acc:.2f}")
print(f"Testing Accuracy: {knn_test_acc:.2f}")
# Colored confusion matrix as a heatmap
cm_knn = confusion_matrix(y_test, knn.predict(X_test_scaled))
plt.figure(figsize=(8, 6))
sns.heatmap(cm_knn, annot=True, fmt='d', cmap='Blues', xticklabels=le.classes_, yticklabels=le.classes_)
plt.title('Confusion Matrix - KNN')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
print("Classification Report:")
print(classification_report(y_test, knn.predict(X_test_scaled), target_names=le.classes_))
```

UNSUPERVISED TECHNIQUE

1. K-MEANS CLUSTERING

```
# K-Means Clustering
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)
df['KMeans_Cluster'] = kmeans_labels

# Visualize K-Means clusters using PM2.5 and AQI
plt.figure(figsize=(8, 6))
sns.scatterplot(x='PM2.5', y='AQI', hue='KMeans_Cluster', data=df, palette='viridis')
plt.title('K-Means Clustering (PM2.5 vs AQI)')
plt.show()
```



```

from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Elbow Method
inertia = []
k_range = range(1, 11) # Try k = 1 to 10

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

# Plot the elbow curve
plt.figure(figsize=(8, 5))
plt.plot(k_range, inertia, 'bo-')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia')
plt.title('Elbow Method For Optimal k')
plt.grid(True)
plt.show()

```

2. HIERARCHIAL CLUSTERING

```

# Hierarchical Clustering
hierarchical = AgglomerativeClustering(n_clusters=3)
hierarchical_labels = hierarchical.fit_predict(X_scaled)
df['Hierarchical_Cluster'] = hierarchical_labels

# Visualize Hierarchical clusters
plt.figure(figsize=(8, 6))
sns.scatterplot(x='PM2.5', y='AQI', hue='Hierarchical_Cluster', data=df, palette='viridis')
plt.title('Hierarchical Clustering (PM2.5 vs AQI)')
plt.show()

```

3. DBSCAN

```
# Step 1: Scale numeric features from the sample
X_sample = df_sample.select_dtypes(include=np.number)
scaler = StandardScaler()
X_sample_scaled = scaler.fit_transform(X_sample)

# Step 2: Compute k-distance graph
min_samples = 5
neighbors = NearestNeighbors(n_neighbors=min_samples)
neighbors_fit = neighbors.fit(X_sample_scaled)
distances, _ = neighbors_fit.kneighbors(X_sample_scaled)
distances = np.sort(distances[:, min_samples - 1])

# Step 3: Automatically find the elbow (knee) point as optimal eps
knee = Kneelocator(range(1, len(distances)+1), distances, curve='convex', direction='increasing')
eps_value = distances[knee.knee] if knee.knee is not None else 0.3 # fallback value

# Plot k-distance graph with detected knee
plt.figure(figsize=(8, 5))
plt.plot(distances, label='k-distance')
plt.axvline(knee.knee, color='r', linestyle='--', label=f"Knee at {knee.knee}, eps = {eps_value:.3f}")
plt.title("K-Distance Graph with Knee Point (Optimal eps)")
plt.xlabel("Points sorted by distance")
plt.ylabel(f"Distance to {min_samples}th nearest neighbor")
plt.legend()
plt.grid(True)
```

```
# Step 4: Run DBSCAN with detected eps
dbscan = DBSCAN(eps=eps_value, min_samples=min_samples)
dbscan_labels = dbscan.fit_predict(X_sample_scaled)
df_sample['DBSCAN_Cluster'] = dbscan_labels

# Step 5: Visualize clustering with less overlap (opacity + point size)
plt.figure(figsize=(8, 6))
sns.scatterplot(
    x='PM2.5', y='AQI', hue='DBSCAN_Cluster',
    data=df_sample, palette='tab10', legend='full',
    alpha=0.6, s=50 # reduce size to avoid overlap
)
plt.title(f'DBSCAN Clustering on Sample (eps={eps_value:.3f}, min_samples={min_samples})')
plt.xlabel('PM2.5')
plt.ylabel('AQI')
plt.legend(title="Cluster", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

4. PCA

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Elbow Method
inertia = []
k_range = range(1, 11) # Try k = 1 to 10

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

# Plot the elbow curve
plt.figure(figsize=(8, 5))
plt.plot(k_range, inertia, 'bo-')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia')
plt.title('Elbow Method For Optimal k')
plt.grid(True)
plt.show()
```

```
from sklearn.decomposition import PCA
import seaborn as sns

# PCA again just to ensure updated visuals
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
df['PCA1'] = X_pca[:, 0]
df['PCA2'] = X_pca[:, 1]

# Plot with KMeans clusters
plt.figure(figsize=(8, 6))
sns.scatterplot(x='PCA1', y='PCA2', hue='KMeans_Cluster', data=df, palette='tab10')
plt.title(f'PCA Visualization with {optimal_k} KMeans Clusters')
plt.show()
```

5. FOLIUM MAP

```
location_coords = {  
    'alipur': [28.7975, 77.1506],  
    'anand vihar': [28.6505, 77.3150],  
    'ashok vihar': [28.6923, 77.1841],  
    'bawana': [28.8182, 76.9784],  
    'dr. karni singh shooting range': [28.4975, 77.2526],  
    'dtu': [28.7499, 77.1170],  
    'dwarka-sector 8': [28.5733, 77.0684],  
    'ito': [28.6262, 77.2426],  
    'jahangirpuri': [28.7326, 77.1706],  
    'jawaharlal nehru stadium': [28.5826, 77.2332],  
    'major dhyan chand national stadium': [28.6128, 77.2301],  
    'mandir marg': [28.6394, 77.2032],  
    'mundka': [28.6789, 77.0313],  
    'najafigarh': [28.6086, 76.9797],  
    'narela': [28.8523, 77.0882],  
    'nehru nagar': [28.5629, 77.2865],  
    'okhla phase-2': [28.5193, 77.2844],  
    'patparganj': [28.6280, 77.2955],  
    'punjabi bagh': [28.6687, 77.1272],  
    'pusa': [28.6370, 77.1536],  
    'rk puram': [28.5647, 77.1830],  
    'rohini': [28.7386, 77.0906],  
    'sirifort': [28.5487, 77.2166],  
    'sonia vihar': [28.7334, 77.2757],  
}
```



```

import folium
import pandas as pd
df_clean = df.dropna(subset=['lat', 'lon', 'AQI']).copy()
df_delhi = df_clean[(df_clean['lat'] >= 28.40) & (df_clean['lat'] <= 28.90) &
                    (df_clean['lon'] >= 76.80) & (df_clean['lon'] <= 77.40)].copy()

def get_color(aqi):
    aqi = float(aqi)
    if aqi <= 50:
        return 'green'
    elif aqi <= 100:
        return 'lightgreen'
    elif aqi <= 200:
        return 'orange'
    elif aqi <= 300:
        return 'red'
    elif aqi <= 400:
        return 'purple'
    else:
        return 'maroon'

```

```

# Create static map centered on Delhi
static_map = folium.Map(location=[28.6139, 77.2090], zoom_start=11, width='80%', height='70%')
# Add AQI points as circle markers
for _, row in df_delhi.iterrows():
    folium.CircleMarker(
        location=[row['lat'], row['lon']],
        radius=6,
        popup=f"{row['StationName']}<br>AQI: {row['AQI']}",
        color=get_color(row['AQI']),
        fill=True,
        fill_opacity=0.7
    ).add_to(static_map)

# Save to HTML
static_map.save('/content/static_aqi_map_delhi.html')

# Download from Colab
from google.colab import files
files.download('/content/static_aqi_map_delhi.html')

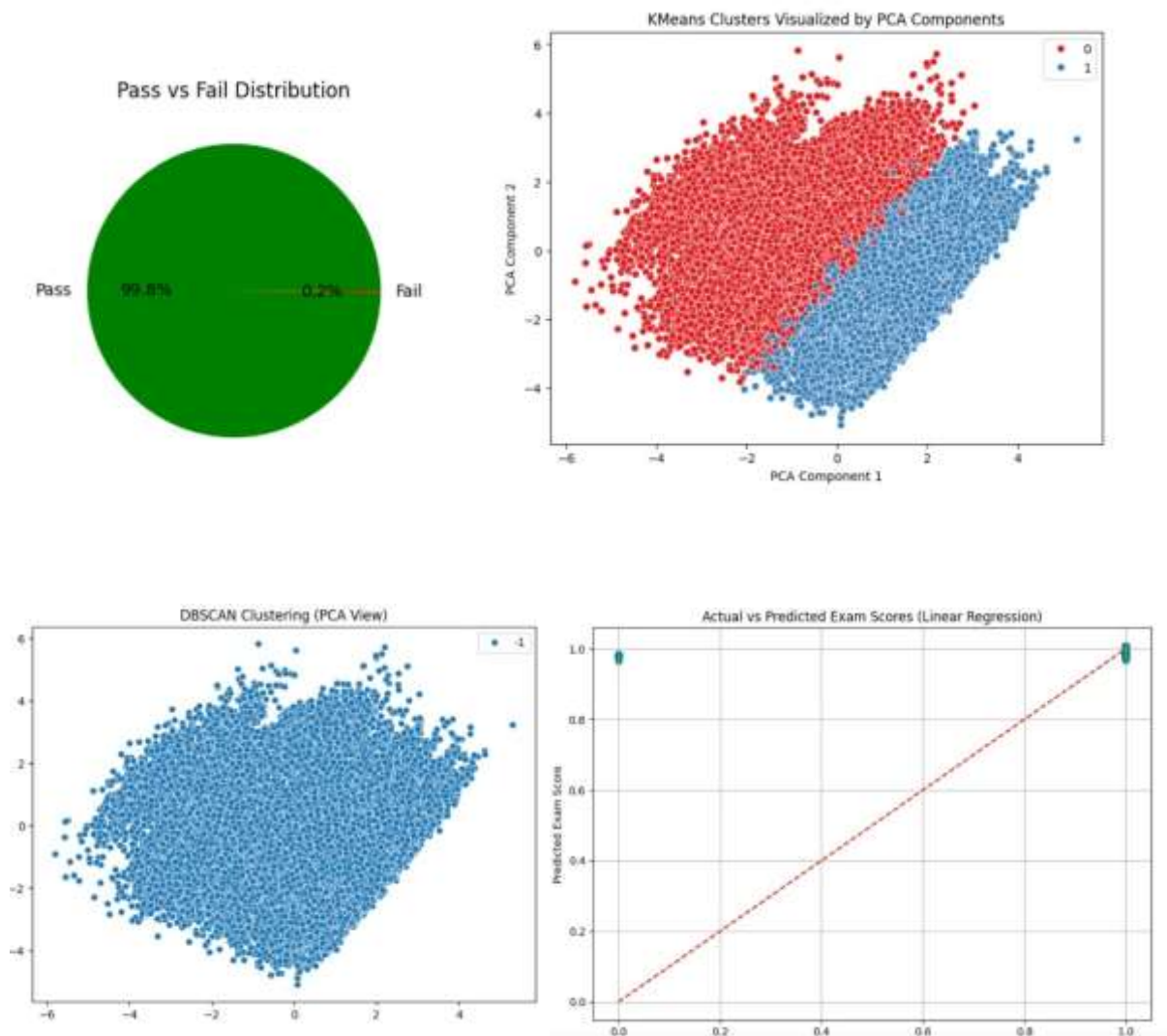
```

During the early phase of experimentation, we utilized the Student Enhanced Habit Dataset, a synthetically generated dataset comprising approximately 80,000 records. The dataset included behavioral, academic, and environmental attributes intended to simulate student lifestyle patterns and their influence on cognitive and health metrics.

Although initial results showed promising accuracy during training, the model exhibited significant overfitting, with a steep drop in performance on validation and test data. This was

primarily attributed to the dataset's synthetic nature, which lacked the variance and noise typically present in real-world data. Consequently, the model failed to generalize effectively.

Due to these limitations, we decided to discontinue this dataset for final modeling and shifted focus to real-world environmental datasets (e.g., Delhi Air Quality Data) for more robust, scalable, and reliable model development.



A. HTML Frontend for Student Dropout Risk Prediction

This frontend was developed to allow users (students or administrators) to input academic performance, attendance, and behavioral habits. The model predicts the risk of dropout based on these features. The interface includes:

- Input form for student attributes (e.g., grades, sleep hours, activity patterns)
- Submission button to get real-time dropout risk prediction

- Display of prediction results (e.g., "Low Risk", "High Risk") using basic HTML and CSS
- Used: SVM (99.83125% accuracy)

Previous GPA (0-4):	3
Semester (1-8):	7
Stress Level (1-10):	3
Screen Time (Hours per Day):	2
Motivation Level (1-10):	9
Exam Anxiety Score (1-10):	9
Time Management Score (1-10):	9
Exam Score (0-100):	85
Gender:	Female

Major:

Computer Science

Part-Time Job:

Yes

Diet Quality:

Poor

Parental Education Level:

Some College

Internet Quality:

High

Extracurricular Participation:

Yes

Social Activity:

Yes

Study Environment:

Co-Learning Group

Access to Tutoring:

Yes

Family Income Range:

High

Family Income Range:

High

Parental Support Level:

Low

Learning Style:

Reading

Predict Dropout Risk

Dropout Risk: No

The image shows a web form with the following fields and values:

- Social Activity:** Yes
- Study Environment:** Co-Learning Group
- Access to Tutoring:** Yes
- Family Income Range:** High
- Parental Support Level:** Low
- Learning Style:** Reading

Below the input fields is a blue button labeled "Predict Dropout Risk". Below the button is a pink box displaying the result: "Dropout Risk: Yes".

[file:///C:/Users/Kalyani%20Agarawal/AppData/Local/Microsoft/Windows/INetCache/IE/BEW8NSMJ/aimlpro\[1\].html](file:///C:/Users/Kalyani%20Agarawal/AppData/Local/Microsoft/Windows/INetCache/IE/BEW8NSMJ/aimlpro[1].html)

HTML Frontend for AQI Prediction (Linear Regression Model)

This frontend interface was created using pure HTML to simulate the AQI prediction functionality. Users can input **date, time, and location details**, and the interface displays the **predicted Air Quality Index (AQI)** generated using a **pre-trained linear regression model**.

The frontend is static and not connected to a backend; it is designed to **accept inputs and demonstrate the output display flow** of the prediction system. This allows a user-friendly mockup to visualize how the final deployed system would present AQI results in a real-time application.

AQI Predictor Login

AQI Prediction

Dwarka

IGI Airport Terminal-3

IHBAS

Jahangirpuri

Janakpuri

Jasola

Lodhi Road

Maharani Bagh

Mandir Marg

Mundka

NSIT Dwarka

Najafgarh

Narela

North Campus DU

Okhla Phase-2

Patpargani

Anand Vihar

AQI Prediction

Date & Time:

dd-mm-yyyy --:--

Predict

Predict AQI

Station Name:

Janakpuri

Date & Time:

06 - 06 - 2025 12 : 05

Predict

**Predicted AQI for Janakpuri at
2025-06-06 12:05 is: 249**

file:///C:/Users/Kalyani%20Agarawal/Documents/ainew.html